

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	Hi! I am Mohit Ruwatia, an open-source enthusiast who has been contributing to Oppia for over a year. I was fascinated by open-source projects during my school days, but I struggled to get the right guidance. That changed when I joined Oppia, where I found a welcoming community that helped me grow both technically and personally. It is the first organization that I started contributing to and I've learned so many invaluable skills related to coding and technology like debugging but also non technical skills like project and team management, effective communication and accountability. I'd like to improve upon these skills further and help the organization in the process by working on a project for GSoC. Being raised in an education and learning focused environment, I resonate deeply with Oppia's mission of providing free education to all. Knowing that my contributions on this project will directly impact millions of learners worldwide is incredibly fulfilling and satisfying and makes this the perfect opportunity.	
What do you hope to learn/achieve during GSoC?	I'd like to learn how to work on a full stack feature with datastores. How to change predefined structures based on the project's needs. I also hope to become a better communicator and planner.	
Which Oppia teams have you collaborated with, and what have you done on those teams?	LaCE	Reviewed PRs, conducted weekly syncs, onboarded new members
	Angular Migration	Helped fix flakes
Contact information	Email: mohitruwatia2312@gmail.com Phone: +91 9892005001	
Preferred method of communication	Email	
Social media handles (optional) <i>This field is optional. We ask for it so that we can tag you in GSoC-related posts if your application is accepted!</i>		
Which timezone will you primarily be in during the summer?	IST (GMT+5:30)	

(If you are a student) When are your school holidays?	May 20 - July 20
What other obligations might you need to work around during the summer?	My college classes will start on July 14th after my summer vacation ends. I have received a No Objection Certificate from my college to work on my project. Certificate . I'll be resuming college by 5/10 August depending on how confident I am regarding the status of my project.
Planned time commitment	During my summer break: 6 hours per day/30 hours per week. After my summer break: 2 to 2.5 hours per day/18 hours per week. I'll be having classes for 6 hours + 2 hours for my daily commute and we'll only be having 3 to 4 working days in a week so I'll be able to manage the time. I'll also utilize one of the weekends if I feel that progress is slow.

Project Timeframe

Note: Oppia will only be offering a single GSoC coding period timeframe this year, starting on **Jun 2**. All work for Milestone 1 must be completed and submitted by **Jul 4** for internal feedback (with any revisions due by **Jul 18**), and all work for Milestone 2 must be completed and submitted by **Aug 27** for internal feedback (with any revisions due by **Sep 10**). We will not be able to extend these deadlines.

Coding period	I will adhere to the above deadlines.
----------------------	---

Communication Channels

Note: The Oppia team places a high emphasis on communication, and we have found that daily contact between contributors and mentors is important for helping keep projects on track. This is why we ask that contributors send short daily updates to their mentors explaining what they have done, where they are stuck, and what they plan to do next.

I can commit to sending daily updates to my mentor by email, each day I work during the GSoC period.	Yes
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	Daily updates on the mail, weekly syncs with the LaCE Team and mentor via google meet and other communication via google chat

Contributor Briefing

Note: A couple of days after the selection results are announced, there will be a mandatory contributor briefing for accepted contributors. Please confirm whether you will be able to attend the briefing.

If accepted, I will be able to attend	Yes
--	---

the contributor briefing at 3 pm UTC on May 10.

Section 2: About Your Project

Project Details

Project title <i>(should match the one on the Project Ideas list)</i>	Clean up the structure for study guides and worked examples
Project size <i>(should match the options on the Project Ideas list)</i>	Large (~350 hours)
Why did you choose this project?	This project has a lot of work relating to the datastore which aligns well with the areas I'd like to work on. I also like that I'll be able to create an RTE component which could be used in some of the RTEs on the Oppia site. It also includes improving worked examples which deeply resonates with my own educational experiences. I found solved problems played a great role in me understanding many of the concepts that I still apply to this day. I think these would be of great benefit to the learners.

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc.¹	Story Editor Migration 404 Console Error State name input flake
I can debug and fix CI failures.²	YouTube Player flake on CI for Exploration Editor: Code to prevent this flake
I can make changes to	

¹ To develop this skill: Tackle an issue where the root cause is unclear, identify the root cause in a comment on the issue thread (perhaps with a debugging doc explaining how you figured it out) and submit a PR that fixes it. Fixing CI failures also counts. You might find [this tutorial](#) helpful.

² To develop this skill: See issues labelled "[CI breakage](#)" on GitHub, or look at [CI failures in develop](#) (or  signs [here](#)) and figure out how to fix them. [This tutorial](#) (for Web) might also be useful.

GitHub Actions workflows. ³	
WEB	
I can write Python code with unit tests. ⁴	Added 405 status code error handling: • Python code • Unit tests Skill Editor page migration: • Python code • Unit tests
I can write TS + Angular code with unit tests. ⁵	Enabled autoloading of MathJax on the required pages: • Typescript + Angular Code • Unit tests
I can write or modify acceptance tests. ⁶	Exploration Editor Page Migration: • Acceptance Test spec file • Acceptance Test utility file
I can write or modify Beam jobs , with tests. ⁷	Beam job to count and display skills that are hanging prerequisites
I can figure out reproduction steps based on information given in the server logs. ⁸	

(Optional) Oppia Community References

Who you worked with	Context in which you worked with them
Hardik Goyal	Serving as the Co-leads of the LaCE Team

³ To develop this skill: Please see the list of issues for the [dev workflow team](#). However, note that this skill is not needed for most projects and there are limited issues available for it. If this affects your project, [please contact the org admins](#).

⁴ To develop this skill: Some [LaCE](#) and [Contributor Dashboard](#) issues have a backend component, and many [Dev Workflow](#) issues do too. Focus on issues that aren't marked as "backlog". This is also a good issue to gain experience with backend tests: [#21308](#).

⁵ To develop this skill: Most [LaCE](#) and [Contributor Dashboard](#) issues have a frontend component. Focus on issues that aren't marked as "backlog". For UI demonstrations, you can tackle a non-backlog responsiveness issue from the [LaCE team](#). You might also find [this tutorial](#) helpful.

⁶ To develop this skill: Solve part of issue [#21646](#) by covering the CUJs for one or more sets of users.

⁷ To develop this skill: Solve issues that are labelled with "[Beam jobs](#)". For information about how these jobs work, please see this [wiki page](#). Jobs should be submitted for testing by following [these instructions](#). Note that, in some cases, you might need to run an audit job to get information about what exactly the issues are on the server before you can create a fix-up job. You might also find [this tutorial](#) helpful.

⁸ To develop this skill: Pick any issue tagged with "[server error](#)", and leave a comment in the issue with a list of reproduction instructions for it, together with video proof that the instructions result in the server error. You do not need to solve the issue in order to demonstrate this skill; the comment above suffices. [This tutorial](#) might help you get started.

Section 3: Proposal Details

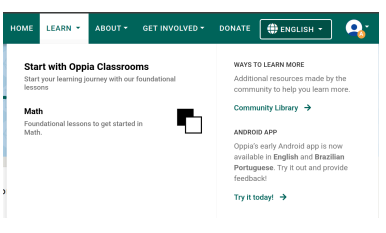
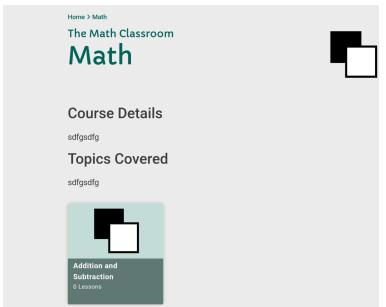
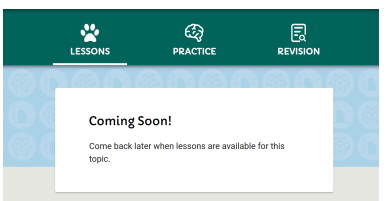
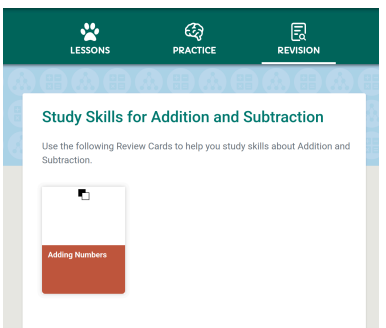
Note: This section is largely adapted from [Oppia TDD Template \(Dec 2024\)](#) -- PLEASE MAKE A COPY . We ***strongly recommend reading that doc***, since it has detailed explanations of how to fill out the various sections. It also contains links to sample TDDs, so that you can see what others have done in the past.

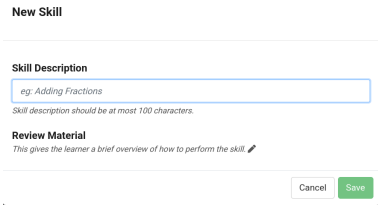
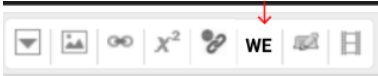
Problem Statement

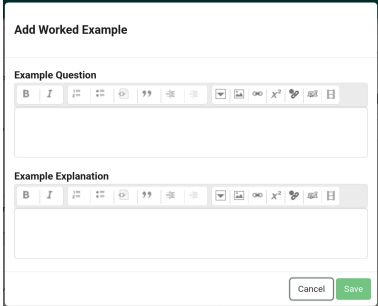
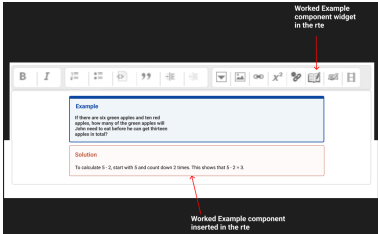
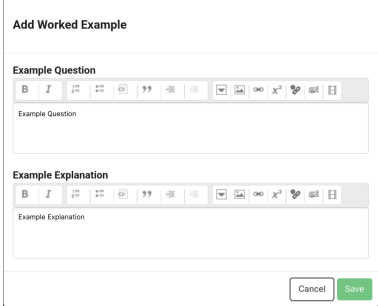
Target Audience	Learners: The primary target users for the Oppia learning platform are learners between the ages of 7 and 14. They may be accessing Oppia on their own or due to a recommendation from a parent or teacher. They may have internet access at home, but it is limited by data. They may also access Oppia through an Android device due to a lack of laptop devices at home. As students get older, between the ages of 12 and 14, they become more independent and self-reliant in finding solutions to problems on their own. They are also more comfortable using technology and the Internet to complete their schoolwork. Lesson creators: They create lessons that will be delivered directly to students facing education inequality across the world. references: PRD
Core User Need	Learners - <i>[Needs based on the learner preferences from the UXR conducted.]</i> - Need more structured study guides to help them stay focused. - Need worked examples inside refresher concept cards before the start of new chapters in a way that showcases and breaks down the core touchpoints of the lessons covered in the previous chapter. - Need worked examples inside concept cards for portions of lessons that are challenging (questions that they get stuck on and/or can't answer correctly) in a way that is clear and helpful in solving the problem they are facing. Lesson creators - <i>[Needs based on the changes required from the UXR conducted.]</i> - Need to know how to place worked examples inside concept cards to make concept card suggestions consistent. references: PRD

Section 3.1: WHAT

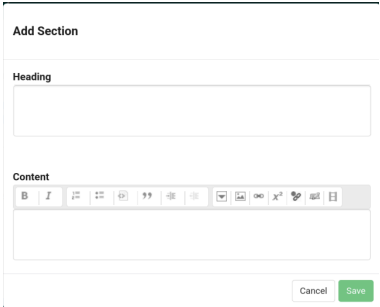
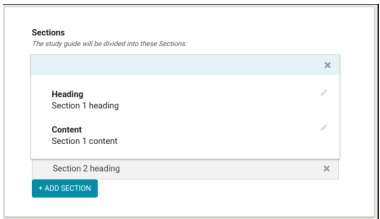
Key User Stories and Tasks that will be implemented

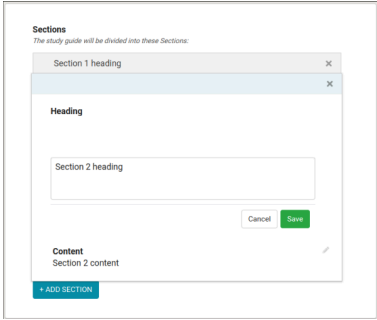
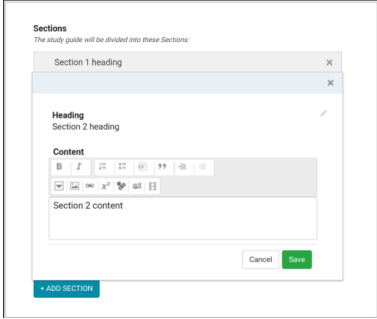
#	Title	User Story Description (role, goal, motivation) "As a ..., I need ..., so that"	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
1	View updated study guide with worked examples	As a learner, I need to be able to access the updated study guide, so that I can revise my concepts. Reference: Issue	Navigate to a classroom.	
			Click on a topic card.	
			Click on the Revision Tab	
			Click on a subtopic card.	
			View study guide with worked examples	Final Study Guide the learner sees (NOTE for future: Make

				the study guide look exactly like this for demos and PR proofs. Do not make any changes to the background colour) References: Components from the Oppia Study Guide Figma Mocks
2	Worked examples integrated with concept cards (at the start of the chapter)	As a learner, I need worked examples at the start of each chapter, so that I can have a reminder of the previous chapter	Start a chapter	Mocks
			See the start of the chapter page with a reminder description and a concept card	Mocks
			Click on the concept card	Mocks
			Scroll through the concept card and find worked examples	Mocks
3	Worked examples integrated with concept cards (suggested when mistakes are made)	As a learner, I need worked examples inside concept cards, so that I can have a longer explanation for problems (it helps me understand how to solve the problem)	Make a mistake on a question	Mocks
			Click on a concept card that is suggested	Mocks
			Scroll through the concept card and find worked examples	Mocks
4	Add worked examples to skill explanation	As a creator, I need to be able to add and edit worked examples to skill explanations (using the rte), so that I can explain how to perform the skill clearly.	Create a skill by clicking on the create skill button	Create new skill modal opens 
			Open the rte to add Review Material.	The worked example rte component widget should be available 
			Click on the WorkedExample rte widget.	Add worked example modal opens

				
			Add a worked example rte component.	The worked examples should be inserted in the rte.  
			Open the skill editor	N/A
			Edit the Review Material of the skill	N/A
			Click on the worked example rte component.	The previous values should already be filled 
			Edit the worked example rte component.	N/A
5	Create a study guide with multiple sections	As a creator, I need to be able to create a study guide with sections and add worked examples to	Create a Subtopic by clicking on the create subtopic button	Create new subtopic modal opens with separate heading and content fields (Currently, for the initial creation of subtopics, even if all the fields

and add worked examples to them. Preview, edit and delete the sections as needed.	them, so that the learner can stay focused while going through the study guide and understand the concepts completely.		in the modal are filled, we do not save it in the datastore. Refreshing at this point would make you lose the data and you'd have to start the creation process again. We expect the user to add a skill through the editor page and save it, only after which we create a datastore entry. Refer to this comment for more context) Create New Study Guide Title* Enter a brief description for the subtopic that students can easily understand, e.g. 'Adding Fractions' Study Guide title should be at most 64 characters. Enter the url fragment for the subtopic* Enter url fragment for the subtopic. The subtopic URL fragment is used to uniquely access the subtopic viewer page. It should consist of one or more hyphen-separated words, all in lowercase, with at most 25 characters in total. Please use meaningful keywords, and avoid using words like 'and', 'of', or 'the'. This subtopic can be accessed at the following URL: localhost/learn/null/asdfasdfsd/revision/ Explanation of the Study Guide* A section of the study guide. More sections can be added using the study guide editor after clicking on the Create Study Guide button. Heading* Enter heading of the section Content* Rich text editor toolbar with icons for Bold, Italic, Text Color, Background Color, Bulleted List, Numbered List, Indent, Outdent, Link, Unlink, Image, Video, Table, and Full Screen. Thumbnail Image* Placeholder for a circular thumbnail image with a question mark and 'Add image' text. Cancel Create Study Guide
		Open the rte to add content to the section	The worked example rte component widget should be available Rich text editor toolbar showing icons for Bold, Italic, Text Color, Background Color, Bulleted List, Numbered List, Indent, Outdent, Link, Unlink, Image, Video, Table, and Full Screen. A red arrow points to the 'WE' (Worked Example) icon.
		Add worked example using the rte widget	The worked examples should be inserted in the rte Rich text editor showing a worked example inserted. The example text is: 'Example: If there are 10 green apples and 5 red apples, how many of the green apples will you need to eat before you can get brown apples in total?' The solution text is: 'The solution is 1-2, start with 5 and count down 2 times. This shows that 5-2=3.' Worked Example component inserted in the rte

			Open subtopic editor	N/A
			Click on add section button	Section mat card with add section button should be present 
			Use the rte to add content to the section	The add section modal with two fields heading and content should open 
			Add worked example using the rte widget	The section tiles should be added when a section is created and removed when a section is deleted 
			Click on any section tile to preview it	Only 1 section will be previewable at a time 
			Click on the heading or content field to edit it	Either the heading or content can be edited at once but not both together

				 
--	--	--	--	--

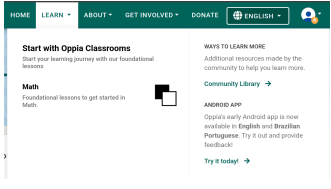
Reference: [PRD](#)

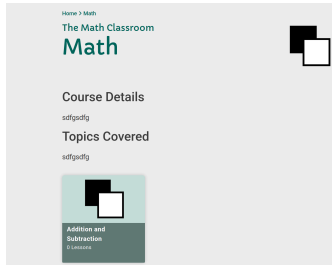
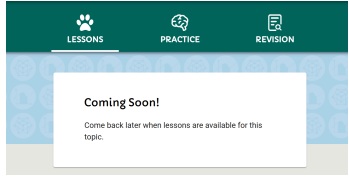
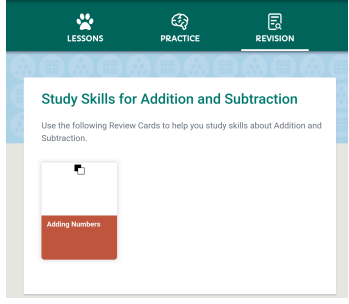
Testing Plan

Critical user journeys

All the points at which screenshots for the acceptance test are to be taken have a  icon.

Milestone 1

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
Logged in user	As a learner, I need to be able to access the updated study guide so that I can revise my concepts. Reference: Issue	Log in as a Curriculum Admin and Set up a Classroom titled 'Math' with a topic titled 'Addition and Subtraction'. The topic should have a study guide called 'Adding	Click on 'Learn' in the navbar.	A dropdown with the 'Math' Classroom should appear. 
			Click on the 'Math' Classroom.	The Math classroom page should open with a topic

		Numbers' with two sections. • Sign In with the email id: 'user@hello.com'. Set username as 'user'.		card titled 'Addition and Subtraction'. 
			Click on the 'Addition and Subtraction' Topic card.	The topic page should open with the 'Lesson Tab' selected by default. There should be a card with the title 'Coming Soon!'. 
			Click on the 'Revision' Tab.	A study guide card titled 'Adding Numbers' should appear 
			Click on the 'Adding Numbers' study guide.	There should be 2 sections in the study guide. Section 1: Heading: 'Adding With Your Fingers' Content: 'One way to add is using your....' Section 2: Heading: 'Using an Addition Table' Content: 'To add two single-digit....'  Final Study Guide the learner sees

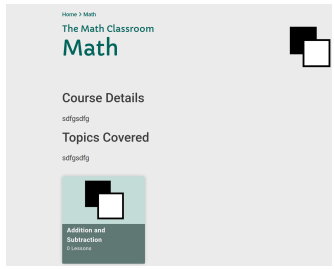
				References: Components from the Oppia Study Guide Figma Mocks
Curriculum Admin	As a creator, I need to be able to create a study guide with multiple sections to make it easier for the learners to focus.	• Sign in as the curriculum admin. • Create a Math classroom with a topic called 'Addition and Subtraction'. Add a diagnostic test skill called 'Diagnostic Test Skill' and a skill to the topic called 'Addition'.	Go to '/topics-and-skills-dashboar d'	The topics and skills dashboard should appear.
			Click on the topic 'Addition and Subtraction'.	The topic editor page should open.
			Click on the 'Add study guide' Button.	The 'Create New study guide' modal should appear.
			Fill in all the details of the study guide. Title: "study guide1", Url fragment: "abcd" Explanation of the study guide: heading: abcd, content: 1234567 thumbnail: send_backward.svg	The Create study guide button should become clickable
			Click 'Create study guide'	The study guide editor page should open. The section mat card should already have 1 tile with heading study guide1 the delete section button  should not be present 
			Click on the 'Add Section' Button under the 'Section' Part.	The add section modal should appear
			Fill the new heading as "Section heading"	Section heading should show up in the rte
			Fill the content as "Section content"	Section content should show up in the rte
			Type "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem.	Length of section exceeded 100 words error should show 

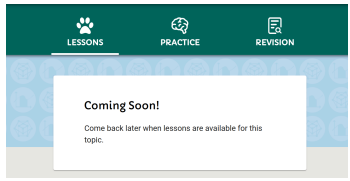
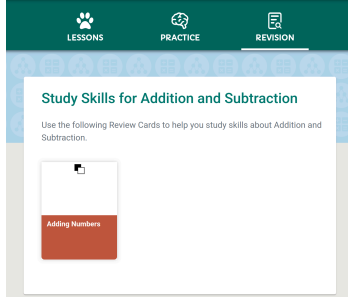
			Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies”	
			Delete “Lorem ipsum dolor sit amet,....”	Only Section content should remain
			Click on the save button of the add section modal	The section mat card should now have 2 tiles with the delete section button  
			Click on the ‘Add Section’ Button under the ‘Section’ Part.	The add section modal should appear
			Fill the new heading as “Section 2 heading”	Section 2 heading should show up in the rte
			Fill the content as “Section 2 content”	Section 2 content should show up in the rte
			Click on the save button of the add section modal	The section mat card should now have 3 tiles with the delete section button 
			Click on the 1st tile	The tile should expand with a preview of the heading and content 
			Click on the 3rd tile	The 1st tile should collapse and the 3rd tile should expand with a preview 

			Click on the heading edit icon	The heading should become editable 
			Click on the content edit icon	The heading should change back to preview and the content should become editable 
			Click on the delete section  button on the middle tile	The delete section modal should open
			Click on Delete Section	The section mat card should only have 2 tiles. (The top tile should be titled "abcd" and the bottom one "Section 2 heading")
			Click on the 'Save Changes' button.	The save draft modal should appear.
			Click on the 'Save Draft' button.	The 'Changes Saved' message should appear.
			Preview the study guide	The study guide has 2 sections: abcd with content 1234567 displayed first and Section 2 heading with content Section 2 content displayed second. 

Milestone 2

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
Logged in user	As an exploration creator, I need to be able to add a concept card (having a worked example) to an exploration and preview it	- Sign in as the curriculum admin - Create a skill called 'skill1' with a worked example rte component added to its explanation. - Assign the skill to a topic called 'ABCD'.	Go to '/creator-dashboard'	Creator's Dashboard page should appear
			Click on the create exploration button	The exploration editor page should open
			Click on the card below introduction	Rte should open
			Add 'hello' to the rte and click on the add concept	A modal should open with the title 'Customize this

		Sign In with the email id: 'user@hello.com'. Set username as 'user'..	card widget of the rte	component' with only 1 available skill
			Click on the skill	The skill should be highlighted in green
			Click on 'Done'	The concept card should be added to the rte
			Click on 'Save Content'	Rte should close
			Click on 'Save Draft'	Save Draft modal should appear
			Click on 'Save Draft'	Changes Saved message should appear
			Navigate to preview tab	There should 'hello' with a concept card link
			Click on the concept card link	The concept card should open. It should have a non collapsible worked example. 
	As a learner, I need to be able to access the updated study guide so that I can revise my concepts. Reference: Issue	- Log in as a Curriculum Admin and Set up a Classroom titled 'Math' with a topic titled 'Addition and Subtraction'. The topic should have a study guide called 'Adding Numbers' with two sections. The first section should also have a worked example. Detailed Classroom setup - Sign In with the email id: 'user@hello.com'. Set username as 'user'.	Click on 'Learn' in the navbar.	A dropdown with the 'Math' Classroom should appear. 
			Click on the 'Math' Classroom.	The Math classroom page should open with a topic card titled 'Addition and Subtraction'. 
			Click on the 'Addition and Subtraction' Topic card.	The topic page should open with the 'Lesson Tab' selected by default. There should be a card with the title 'Coming Soon!'.

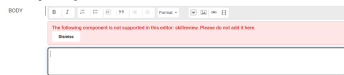
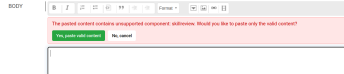
				
			Click on the 'Revision' Tab.	A study guide titled 'Adding Numbers' should appear 
			Click on the 'Adding Numbers' study guide Card.	There should be 2 sections in the study guide. Section 1: Heading: 'Adding With Your Fingers' Content: 'One way to add is using your....' Section 2: Heading: 'Using an Addition Table' Content: 'To add two single-digit....' The first section should also have a worked example which should be in the collapsed state by default.  Final Study Guide the learner sees References: Components from the Oppia Study Guide Figma Mocks
			Click on the worked example solution dropdown arrow.	The solution should expand  Final Study Guide the learner sees References: Components from the Oppia Study Guide Figma Mocks

Curriculum Admin	As a creator, I need to be able to create a study guide with multiple sections containing worked examples to make it easier for the learners to focus.	• Sign in as the curriculum admin. • Create a Math classroom with a topic called 'Addition and Subtraction'. Add a diagnostic test skill called 'Diagnostic Test Skill' and a skill to the topic called 'Addition'.	Go to '/topics-and-skills-dashboard'	The topics and skills dashboard should appear.
			Click on the topic 'Addition and Subtraction'.	The topic editor page should open.
			Click on the 'Add study guide' Button.	The 'Create New study guide' modal should appear.
			Fill in all the details of the study guide. Title: "study guide1", Url fragment: "abcd" Explanation of the study guide: heading: abcd, content: 1234567 thumbnail: send_backward.svg	The Create study guide button should become clickable
			Click 'Create study guide'	The study guide editor page should open. The section mat card should already have 1 tile with heading study guide1 the delete section button  should not be present 
			Click on the 'Add Section' Button under the 'Section' Part.	The add section modal should appear
			Fill the new heading as "Section heading"	Section heading should show up in the rte
			Fill the content as "Section content"	Section content should show up in the rte
			Type "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet	Length of section exceeded 100 words error should show 

			nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies”	
			Delete “Lorem ipsum dolor sit amet,.....”	Only Section content should remain
			Click on the save button of the add section modal	The section mat card should now have 2 tiles with the delete section button 
			Click on the ‘Add Section’ Button under the ‘Section’ Part.	The add section modal should appear
			Fill the new heading as “Section 2 heading”	Section 2 heading should show up in the rte
			Fill the content as “Section 2 content”	Section 2 content should show up in the rte
			Click on the Worked Examples widget of the rte to add a worked example.	The ‘Add Worked Example’ modal should open up.
			Add the example question as “Type the number one” and solution as “1”	The values should show up in the rte
			Click on Save (for worked example)	The modal should close and Worked Example should be added to the rte
			Click on the Worked Examples component of the rte to add a worked example.	The ‘Add Worked Example’ modal should open up.

			Add the example question as "Type the number two" and solution as "2"	The values should show up in the rte
			Click on Save (for worked example)	The modal should close and Worked Example should be added to the rte
			Click on the Worked Examples widget of the rte to add a worked example.	An error message will be displayed stating that only two worked examples are allowed. 
			Delete 1 worked example	Only 1 worked example remains in the rte
			Click on the save button of the add section modal	The section mat card should now have 3 tiles with the delete section button 
			Click on the 1st tile	The tile should expand with a preview of the heading and content 
			Click on the 3rd tile	The 1st tile should collapse and the 3rd tile should expand with a preview 
			Click on the heading edit icon	The heading should become editable 
			Click on the content edit icon	The heading should change back to preview and the content should become editable 
			Click on the delete section  button on the middle tile	The delete section modal should open
			Click on Delete Section	The section mat card should only have 2 tiles. (The top tile should be titled "abcd" and the bottom one "Section 2 heading")
			Click on the 'Save Changes' button.	The save draft modal should appear.
			Click on the 'Save Draft' button.	The 'Changes Saved' message should appear.
			Preview the study guide	The study guide has 2 sections: abcd with content 1234567 displayed first and

As a creator, I cannot add worked examples to unsupported rte directly by creating them through the worked examples rte component widget or indirectly by copying them from a supporting rte and pasting them				Section 2 heading with content Section 2 content displayed second.The 2nd section has a worked example added to it. 
			Go to '/topics-and-skills-dashboar d'	The topics and skills dashboard should appear.
			Click on the Skills Tab	The skills tab should have 1 skill with the name Test skill
			Click on Test Skill	Skill editor opens
			Click on the text under Review Material	The Review Material rte should open
			Click on the Worked Example rte component widget	The 'Add Worked Example' modal should open up.
			Add the example question as "Type the number one" and solution as "1"	The values should show up in the rte
			Click on Save (for worked example)	The modal should close and Worked Example should be added to the rte
			Click on the worked example rte component inserted into the rte	The component should be highlighted
			Copy the component	The component should be copied
			Navigate to /blog-dashboard on a new tab	The Add your Author Name and Biography: modal should show up
			Click on the Bio input box and type "Abcd"	The save button should become clickable
			Click on the save button	The modal should close
			Click on the CREATE NEW BLOG POST Button	The blog post editor page should open. In the Body rte there should be only 3 rte component widgets: Image, Link and Video
			Click on the input box of the Body rte	A cursor should show up inside the rte and it should become highlighted
• Sign in as the curriculum and blog admin. • Create a skill called "Test skill"			Video of the expected behaviour for the below steps	

			Paste the component	An error message is displayed:  (TODO: replace the screenshot with one that says workedexample once the component is created)
			Click on the Dismiss button to dismiss the message	The error message disappears.
			Switch to the exploration editor tab	The exploration editor should show up
			Add the text “Sample text” after the worked example component	The text should be added
			Select all the content of the rte and copy it	The content should be copied
			Switch to the blog editor tab	The blog editor should show up
			Click on the rte and paste the copied content	An error message should show up:  (TODO: replace the screenshot with one that says workedexample once the component is created)
			Click on Yes, paste valid content	The error message should disappear and Sample text should be pasted in the rte: 
			Clear the contents of the rte	The rte should be empty
			Paste the copied content	An error message should show up:  (TODO: replace the screenshot with one that says workedexample once the component is created)
			Click on No, cancel	The error message should disappear and no content

As a creator I need to be able to create a skill with worked examples				should be pasted:
			Paste the copied content	
			Type "qwerty"	An error message should show up:  (TODO: replace the screenshot with one that says workedexample once the component is created)
	Sign in as the curriculum admin		Go to '/topics-and-skills-dashboar d'	The topic and skills dashboard should open
			Click on the 'Create Skill' Button.	The skill creation modal should open
			Fill in the skill description as "Skill 1"	Skill 1 should appear in the input box
			Add "hello" in the rte of the 'Review Material' section.	hello should appear in the rte
			Click on the Worked Examples rte component widget to add a worked example.	The 'Add Worked Example' modal should open up.
			Add the example question as "Type the number one" and solution as "1"	The values should show up in the rte
			Click on Save (for worked example)	The modal should close and Worked Example should be added to the rte
			Double click on the worked example rte component inserted into the rte	The add worked example modal should open up with the previous values 
			Click on cancel	The modal should close
			Click on Save (for skill creation)	The skill editor page should open
			Click on the 'Review Material' section	Rte should open
			Click on the Worked Examples component of	The 'Add Worked Example' modal should open up.

			the rte to add a worked example.	
			Add the example question as "Type the number two" and solution as "2"	The values should show up in the rte
			Click on Save (for worked example)	The modal should close and Worked Example should be added to the rte
			Click on Publish Changes	Publish changes modal should open
			Enter description as abcd	abcd should show up in the text field
			Click on Publish Changes	Changes saved message should appear
			Go to the preview tab	The preview tab should open
			Preview the skill/concept card	The skill should have 2 worked examples 

Technical Requirements

Additions/Changes to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	New / Existing	Description of the request/response contract (and, if applicable, how it's different from the previous one). For new endpoints, include which access control decorator (e.g. "can_play_exploration", "open_access", etc.) will be used.
1.	/topic_editor_handler/data/<topic_id>	PUT	Existing	The PUT body data (topic_and_subtopic_page_change_dicts) will be changed to accept the new study guide explanation structure having sections (Explained in this section)
2.	/subtopic_page_editor_handler/data/<topic_id>/<subtopic_id>	GET	Existing	The response returned will be different as it will have the new study guide explanation structure having sections (Explained in this section)
3.	/subtopic_data_handler/<classroom_url_fragment>/<topic_url_fragment>/<subtopic_url_fragment>	GET	Existing	The response returned will be different as it will have the new study guide explanation structure having sections (Explained in this section)

	ment>			
4.	/gettranslatabletexthandler	GET	Existing	Currently only has support for explorations. We will need to add support for study guides as well (Explained in this section)
5.	/concept_card_handler/<selected_skill_ids>	GET	Existing	The response will no longer contain worked_examples (Explained in this section)
6.	/skill_editor_handler/data/<skill_id>	PUT	Existing	Adding, updating and deleting worked examples via the change_dicts as a separate property will be deprecated. It will become a part of updating the skill explanation. (Explained in this section)

New/Changed Calls by the Web Frontend to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	Description of why the new call is needed, or why the changes to an existing call are needed
1.	/subtopic_page_editor_handler/data/<topic_id>/<subtopic_id>	GET	The response returned will be different as it will have the new study guide explanation structure having heading and content. Called from the creator side while editing the study guide (Explained in this section)
2.	/subtopic_data_handler/<classroom_url_fragment>/<topic_url_fragment>/<subtopic_url_fragment>	GET	The response returned will be different as it will have the new study guide explanation structure having heading and content. Called from the learner side when viewing the study guide (Explained in this section)
3.	/gettranslatabletexthandler	GET	Currently only has support for explorations. We will need to add support for study guides as well. Called while fetching text that needs to be translated on the contributor dashboard (Explained in this section)
4.	/concept_card_handler/<selected_skill_ids>	GET	The response will no longer contain worked_examples. Called when viewing a concept card in the exploration preview tab or exploration player. (Explained in this section)

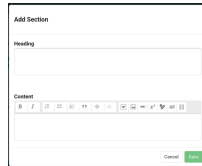
Web Server Storage Model Additions/Changes

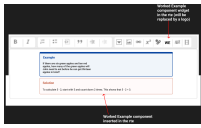
#	Datastore model	Description of changes	How will existing data be handled?
1.	StudyGuideMo	It will be similar to the	No existing data

	del	SubtopicPageModel. • A new property called <code>next_content_id_index</code> will be created that stores the next section id to be assigned to a newly created section. • sections will be created which will be similar to <code>page_contents</code> of SubtopicPageModel (repeated JsonProperty). Structure of sections: <pre>sections: [{ heading: SubtitledUnicode, content: SubtitledHtml }, ...]</pre> • <code>written_translations</code> and <code>recorded_voiceovers</code> will not be present in the sections property. They will be added to models created in the EntityTranslationsModel and EntityVoiceoversModel. Reference: Oppia wiki	
2.	SubtopicPage Model	It will be deprecated once all the data is migrated to the StudyGuideModel and all the domain classes and services have started using the StudyGuideModel.	• The existing html of <code>subtitled_html</code> will be migrated to StudyGuideModel. For the new structure, the heading will be the title of the subtopic and the content will be copied from the original html in <code>subtitled_html</code> • The existing <code>recorded_voiceovers</code> and <code>written_translations</code> do not contain any existing data as of now Reference: Oppia wiki
3.	GeneralTranslationOpportunityModel	It will be implemented by Hasitha as part of his project. TDD . It will be used in this project for the study guide translation opportunities	For all the study guides we will write a beam job to generate the OpportunityModels. Currently no OpportunityModels exist for study guides
4.	SkillModel	worked_examples in the skill_contents	Sean (or anyone with access to

		property will be deprecated.	the datastore) will check the contents of the worked examples of all skills. It should be empty. If so, we can safely deprecate worked_examples. If there are some entries, the skill id and the contents of the worked example will need to be noted down to be re added after the worked examples RTE is created.
4.	SkillSummary Model	worked_examples_count will be deprecated	The existing data won't be useful to us anymore. We will remove this property from all instances

UI Screens/Components

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	Section Mat card	Mat card used to view the currently added sections. Has an add section button to add more. Sections can also be deleted using the x button. On creating a section, its tile will be added to the mat card. On clicking this tile, a preview of the heading and content will show up. To edit, the icon can be clicked to open up the rte.	No	Mock 	Ensure that the components will be fully accessible i.e. the tiles and their edit and delete buttons can be accessed with just a keyboard and have the appropriate aria labels
2.	Section creation modal	Modal to create a section with the Heading and Content fields	No	Mock 	The modal will function completely with just a keyboard. The checkboxes will follow web accessibility guidelines, and proper aria tags will be

					maintained.
3.	New study guides with sections	Study guide with the content divided into sections to make the concepts easier to understand. Would also make translation easier	No	Mock  Reference: Figma mock from project description on Oppia wiki	Should work with screen readers. All buttons should be accessible by just keyboard as well
4.	New worked example rte widget (creator facing)	The rte component widget used to add a worked example and the actual component that gets inserted in the rte	No	Mock 	Button will have proper aria tags
5.	New worked	The rte component that the learner sees while viewing a concept card or	No	Mock	Should work with screen

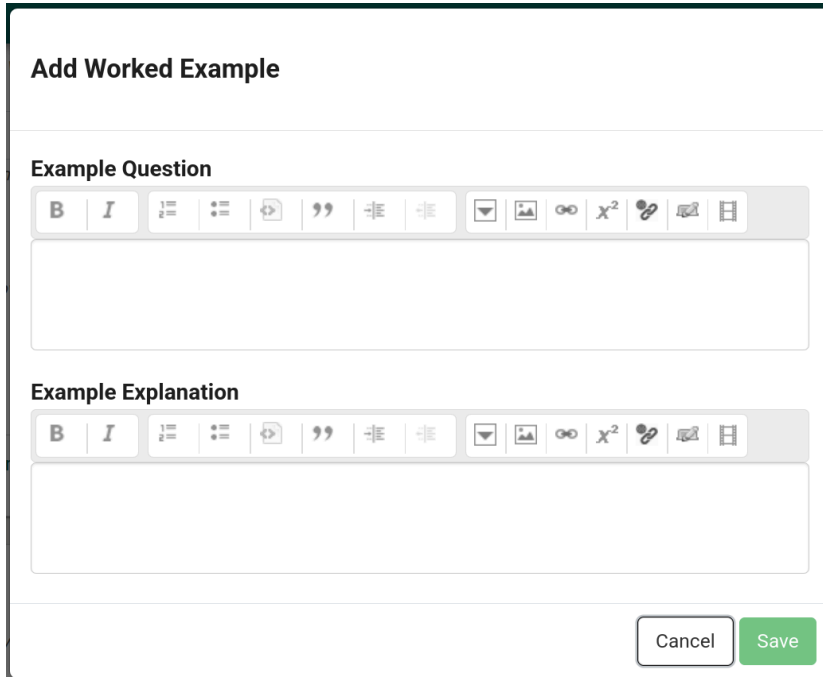
	example compone nt (learner facing)	a study guide	Example Calculate $25 + 29$  Calculate the addition using the number line. Solution Example If there are six green apples and ten red apples, how many of the green apples will John need to eat before he can get thirteen apples in total? Solution Example Calculate $25 + 29$  Calculate the addition using the number line. Solution Step 1: We start from the right, and go towards to the left. • The rightmost digit is 6. It is in the ones place. • The digit 0 is in the tens place. • The digit 5 is in the hundreds place. • The digit 2 is in the thousands place. <table><tr><td>2</td><td>5</td><td>0</td><td>6</td></tr><tr><td>Thousands Place</td><td>Hundreds Place</td><td>Ten Place</td><td>One Place (the right digit from the zero)</td></tr></table> Reference: Figma mock from project description on Oppia wiki	2	5	0	6	Thousands Place	Hundreds Place	Ten Place	One Place (the right digit from the zero)	readers. All buttons should be accessible by just keyboard as well
2	5	0	6									
Thousands Place	Hundreds Place	Ten Place	One Place (the right digit from the zero)									

Data Handling and Privacy

No user data is stored.

Connection to Existing Work

The same Add Worked Examples modal which was used earlier will be used with the new rte component as well.



The image shows a modal titled "Add Worked Example". It contains two text input areas, each with a rich text editor toolbar above it. The first section is labeled "Example Question" and the second is labeled "Example Explanation". Both toolbars include buttons for bold (B), italic (I), bulleted list, numbered list, link, unlink, quote, code, and various alignment options. There are also buttons for inserting a table, image, equation (x^2), and a link icon. At the bottom right of the modal are "Cancel" and "Save" buttons.

Other Requirements

N/A

Section 3.2: HOW

Existing Status Quo

Subtopic Page:

Structure:

The **SubtopicPageModel** has a property called **page_contents** which contains the **subtitled_html** field containing the **content_id** and the **html**. The **written_translations** and **recorded_voiceovers** are also a part of **page_contents**. Current structure:

```
{
  "subtitled_html": {"content_id": "content", "html": ""},
  "recorded_voiceovers": { "voiceovers_mapping": { "content": {} }},
  "written_translations": { "translations_mapping": { "content": {} } }
}
```

This does not follow the standard procedure for storing translations and voiceovers as explorations. A separate **EntityTranslationsModel** and **EntityVoiceoversModel** should be created for each piece of content.

Note: The subtopic page translations and voiceovers should follow a similar structure and workflow to the exploration translations and voiceovers.

Translations and Voiceovers in Exploration editor:

Firstly, the **ExplorationModel** has a **next_content_id_index** property which stores the next content id to be assigned. Whenever some new content is added, to generate a content_id, the **ContentIdGenerator** is used to create a content_id by appending the **next_content_id_index** to the type of content and prefix and assigned. Then **next_content_id_index** gets incremented by 1.

next_content_id_index 2

When a story is published, an **ExplorationOpportunitySummaryModel** is created from the **topic_services.py** file. Its key is the **exploration_id**. This contains all the languages that do not currently have an accepted translation for them.

We use the **ExplorationOpportunitySummaryModel** to populate the contributor dashboard with the exploration translation opportunities based on the **incomplete_translation_language_codes** array and **exploration_id**.

When a user suggests a translation, this content_id is used to create a **GeneralSuggestionModel** and to keep track of what content the translation was submitted for.

```
{
  "cmd": "add_written_translation",
  "content_id": "content_7",
  "state_name": "First State",
  "language_code": "pt",
  "content_html": "<p>Our Solar System consists",
  "translation_html": "<p>asdfasfasdfzxc</p>",
  "data_format": "html"
}
```

When a reviewer accepts a translation, an **EntityTranslationsModel** is created which has a property called translations. The keys of this property are the content_ids of the original entity.

```
{
  "content_7": {
    "content_value": "<p>asdfasfasdfzxc</p>",
    "content_format": "html",
    "needs_update": false
  }
}
```

When a user requests some content in the translated language, we can fetch it from

EntityTranslationsModel using this **content_id**.

Similar flow is followed for voiceovers as well but an **EntityVoiceoversModel** is created instead.

Creation/Updation:

Subtopics can be added from the topic editor page by clicking on the add subtopic button

+ ADD SUBTOPIC

Which opens a 'Create new subtopic modal'. The SubtopicPageModel is created once a subtopic is saved for the first time on the subtopic editor page. The save is done by a PUT request to /topic_editor_handler/data/<topic_id> with the subtopic details passed in the topic_and_subtopic_page_change_dicts of the payload using a [TopicAddSubtopicChange](#) interface. For updating the content, the subtopic editor page can be used. The same endpoint as the creation is used but with different interfaces depending on the type of update.

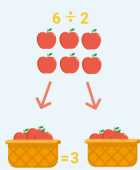
UI:

The users can access the study guide by going to a classroom, clicking on a topic card, clicking on the revision tab and then clicking on the subtopic card. The current ui is a single block of content.

Basic concepts of Division

What is division?

In mathematics, the word "division" means the operation which is the opposite of multiplication. Division is usually used when we need to divide a certain amount of things equally. If you had to distribute 6 apples equally among 2 people, each person would get 3 apples. What you did is called division!



The symbol for division can be a slash, a line, or the division sign.

Example: $6/2$ or $\frac{6}{2}$ or $6 \div 2$ means '6 divided by 2'.

Parts of a division equation

Always remember that:

- The number which we divide is called the **dividend**.
- The number by which we divide is called the **divisor**.
- The result obtained is called the **quotient**.
- The number left over is called the **remainder**.

This means that the dividend is the first number and the divisor is the second number. The answer is expressed as a quotient (possibly with a remainder).

Example: Consider the statement: $13 \div 4 = 3$ with remainder 1.

In this example, 13 is the dividend, 4 is the divisor, 3 is the quotient, and 1 is the remainder.

Remember: In division, the order of the numbers is important. For example, $1 \div 2$ has a different value from $2 \div 1$. The first number must be divided by the second, not the other way around.

Worked Examples:

Structure:

The **SkillModel** has a property called **skill_contents** which contains the **worked_examples** field.

```
{
  ....
  "worked_examples": [
    {
      "question": { "content_id": "worked_example_question_1", "html": "" },
      "explanation": { "content_id": "worked_example_explanation_1", "html": "" }
    }
  ],
  ....
}
```

Creation/Updation:

+ ADD WORKED EXAMPLE

Can be added by clicking the button on the skill editor page. This opens a modal where the example question and answer can be added (This modal can be used for the new worked examples rte component that we will create in this project as well). The skill model is updated with the new worked example when the changes are published.

UI:

Worked Examples can be accessed by adding the skill as a concept card to an exploration and previewing it

Solution Overview

Subtopic Page Restructure:

Model changes:

We will create a new model called **StudyGuideModel** similar to the **SubtopicPageModel** which will have a new property called **sections** and **next_content_id_index** which will be used to assign an id to the **heading** and **content** part of the new **section** property similar to how **next_content_id_index** is used for the **ExplorationModel**. The [ContentIdGenerator](#) class of **translation_domain.py** will be used here similar to how it is used in the **exp_domain.py** file. [Detailed explanation](#)

We will create a **StudyGuideBackendDict** similar to the **SubtopicPageBackendDict** and **StudyGuide** similar to the **SubtopicPage** class in **subtopic-page.model.ts** to reflect the same.

Create a new **StudyGuideSectionsBackendDict** similar to the **SubtopicPageContentsBackendDict** to store the new **heading** and **content**

Also modify the **subtopic_page_domain** file by creating a **StudyGuide** class which will have similar methods to the **SubtopicPage** class but will work with the new **StudyGuideModel** and new structure having **sections**. We will maintain both classes till the migration is complete and make sure that any operations done on a **SubtopicPageModel** object are replicated for the **StudyGuideModel** object as well. Once it is done, we will change the filename to **study_guide_domain** and deprecate all instances of

SubtopicPage making sure that only the **StudyGuide** class and its supporting classes remain. We will also deprecate all instances of **written_translations** and **recorded_voiceovers**.

Each study guide will have at least 1 section i.e. heading and content pair. We will add backend validation checks that will check if the study guide has at least one section. We will not save the study guide if it fails here. The user will not be allowed to delete the last remaining section.

When a study guide is validated and added to a published topic, we will create a [GeneralTranslationOpportunityModel](#). This will be used to store all the opportunities for translations and voiceovers. We will create a temporary form on the admin page to test the study guide translation. This will not be available on prod so we will add it under Admin > Activities which is hidden in prod. The user will be able to access all the study guide translation opportunities from here. They will enter the study guide id, which will be used to populate the opportunities based on the language chosen. They will then be able to submit a translation for these opportunities which will be used to create suggestions. There will also be an option to accept/reject the suggestion so that a model is created for them. This form will be deprecated once translating the study guide is enabled on the contributor dashboard can be added for study guides. The opportunities will be fetched by using the **GeneralTranslationOpportunityModel**. When a suggestion is submitted using the form, we will create a [GeneralSuggestionModel](#) to store it. When it is reviewed and accepted, we will create an **EntityTranslationsModel** for it.

Schema migration:

Perform a schema migration where the **next_content_id_index** will be set to **2** for all the existing subtopic pages (as they will all be having only 1 section with the content ids 0 and 1 being assigned). There will be only 1 section with the **heading** as **the subtopic's title** and the **content** as **the previously existing html content** of the **subtitledhtml** property. The **content ids** will be **section_heading_0** for **all the headings** and **section_content_1** for **all the content**. The **written translations** and **recorded voiceover** fields **should be empty** (will be confirmed by an audit job) so they won't be needing special migration.

Generating study guide opportunities:

We will modify the **TranslationOpportunityGenerationJob** to include study guides as well or implement a new **StudyGuideTranslationOpportunityGenerationJob** using this [logic](#) if the **TranslationOpportunityGenerationJob** hasn't been written by Hasitha yet.

We also have a regenerate opportunities button on the admin page which uses the [regenerate_opportunities_related_to_topic](#) method of **opportunity_services.py**. We will modify it to support study guides as well. We will follow the implementation mentioned [here](#).

Character counting in RTE:

We will implement **character counting logic** in the **content rte** of the newly added **Add Section** part of the study guide. Each section of the current revision cards on Oppia on an average contains 500 to 600 characters. To offer some leeway, we will set a limit of 1000 characters after which no more characters can be typed. The tags of the images added will not contribute to this limit. We will adopt a similar approach to the [hint-editor.component.ts](#) file specifically the [isHintLengthExceeded](#) method which checks the number of characters typed in the editor by using the **htmlLengthService** and displays an error div if the length exceeds the limit.

Frontend changes:

During the initial creation of the study guide by using the Create New study guide Modal, the Explanation of the study guide field will only be able to add 1 section. We will add a note under Explanation stating that this is the first section and more can be added later in the study guide editor. The functionality of being able to add multiple sections in the modal will make the modal messy. It will also be difficult for the user to keep track of the different sections in the modal.

Create New Study Guide

Title*

Enter a brief description for the subtopic that students can easily understand, e.g. "Adding Fractions"

Study Guide title should be at most 64 characters.

Enter the url fragment for the subtopic*

Enter url fragment for the subtopic.

The subtopic URL fragment is used to uniquely access the subtopic viewer page. It should consist of one or more hyphen-separated words, all in lowercase, with at most 25 characters in total. Please use meaningful keywords, and avoid using words like "and", "of", or "the". This subtopic can be accessed at the following URL:
localhost/learn/null/asdfasdfasdf/revision/

Explanation of the Study Guide*

A section of the study guide. More sections can be added using the study guide editor after clicking on the Create Study Guide button.

Heading*

Enter heading of the section

Content*

B

I

Thumbnail Image*

?

Add image

Cancel

Create Study Guide

This should be fine as we will include a [Section Mat card](#) which can be used to add multiple sections (explanations consisting of a heading and content). The Mat card will have a list of all the created sections.

Sections

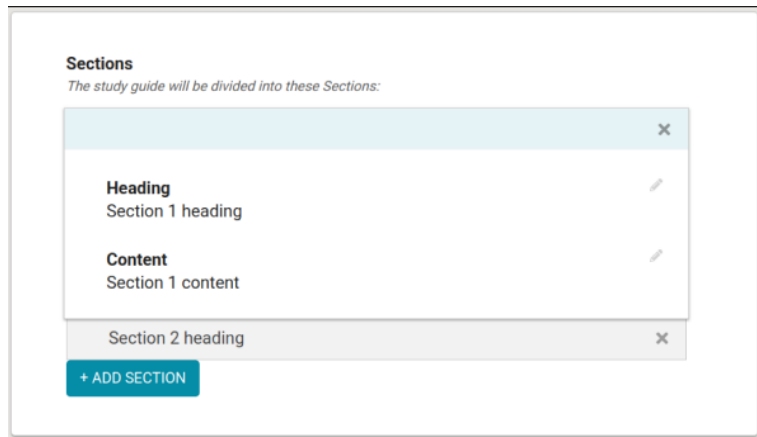
The study guide will be divided into these Sections:

Section 1 heading

Section 2 heading

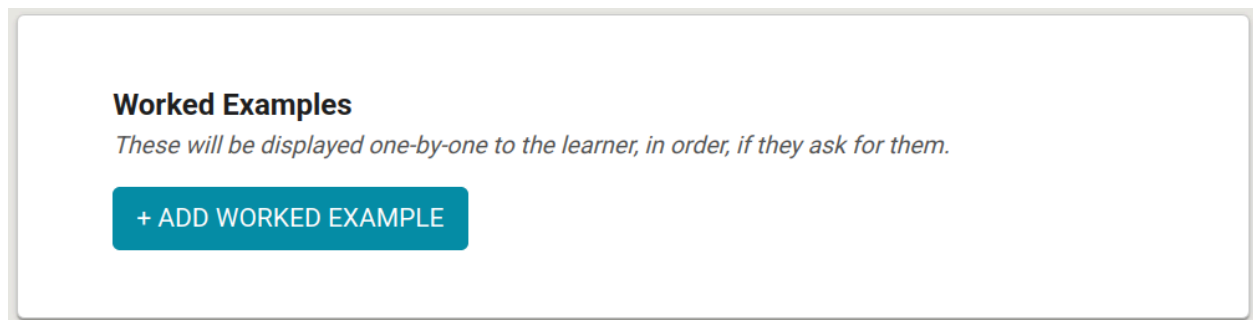
+ ADD SECTION

When the tile of a section in the list is clicked, it will expand and the heading and content fields will be available for preview. At a time only 1 section can be expanded to preview.



The sections can be edited by clicking on the edit icon . At a time either the heading or the content can be edited, but not both at once.

We will be using the worked examples mat card component from the skill editor to create the Sections mat card as it has all the functionality we need for the section cards like seeing all the sections in the form of tiles, clicking on the tiles to expand into a card and show more details, inline editing and ability to delete sections.



The code for this is located in the file [skill-concept-card-editor.component.html](#). It will be removed from the skill editor in milestone 2 as worked examples will be deprecated from the skill editor.

Sections

The study guide will be divided into these Sections:

Section 1 heading

Heading

Section 2 heading

Cancel

Save

Content

Section 2 content

+ ADD SECTION

Sections

The study guide will be divided into these Sections:

Section 1 heading

Heading
Section 2 heading

Content

B I 1/2 3/4 1/2 3/4 [icon] " " =E =E

[icon] [icon] [icon] X² [icon] [icon]

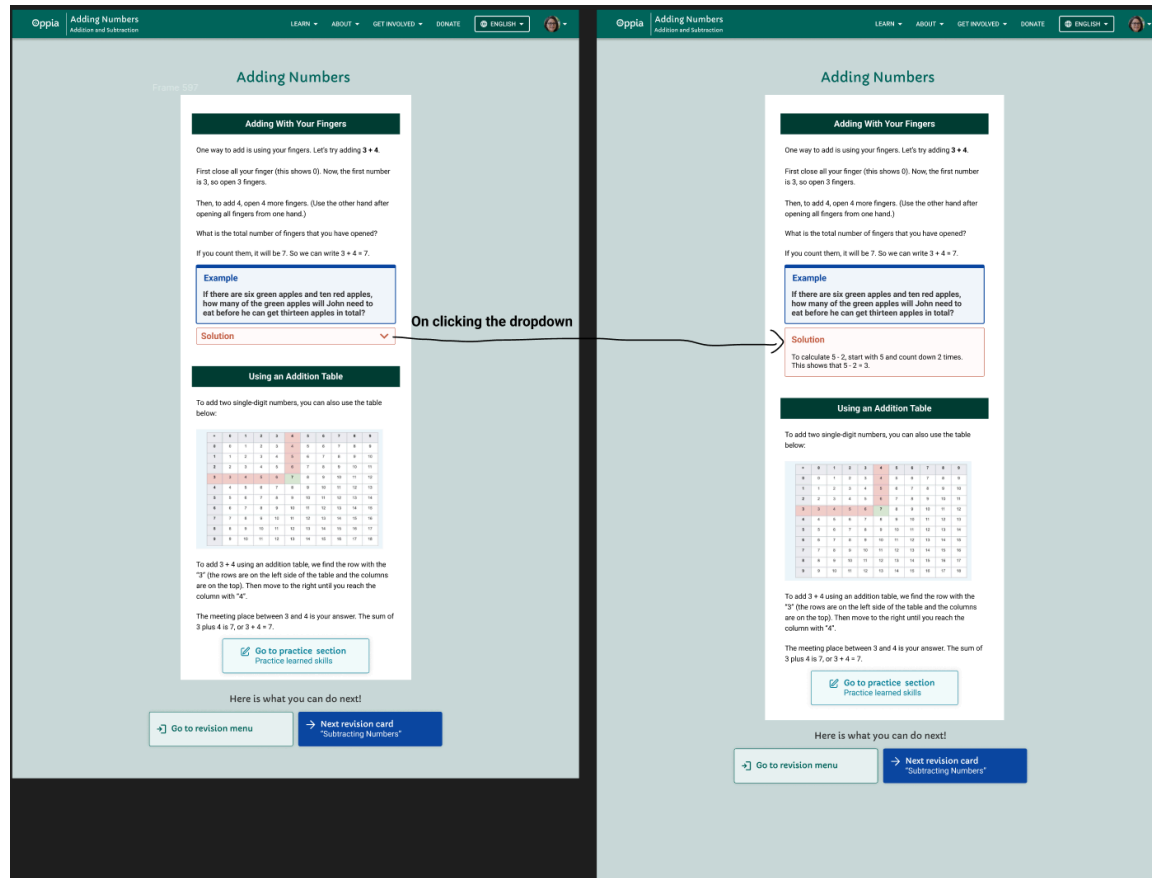
Section 2 content

Cancel Save

+ ADD SECTION

To delete them, a  will be present in the section tiles. We will not show the  button for the last remaining section of a study guide as we will not allow deleting it. The user can still edit it. Basically we will strictly enforce the rule that each study guide should have at least 1 section.

The learner will see the [restructured study guide](#) as shown in the mocks from the Oppia GSoC Project ideas page.



Worked Examples:

Deprecating worked examples:

A beam job will be written to check that the **worked_examples** property of all skills in production is empty. The property will then be removed from the **SkillModel** and deprecated by performing a schema migration. We will also remove the **worked_examples_count** property from the **SkillSummaryModel**. The mat card containing the worked examples will be removed from the ui. The worked example creation modal will be saved and will be used when creating new worked examples using the newly implemented rte component.

New worked examples rte component:

A new folder will be created in **oppia/extensions/rich_text_components/** called **WorkedExample**. This will have a directives folder where the component files will be placed. While registering the component in the **rich_text_components_definitions.ts**, **is_block_element** and **is_complex** will be set to **true**.

To display the rte component only on the skill editor and study guide editor pages, we will introduce a new property called **rte_components** in the UiConfig interface of the **ck-editor-4-rte.component.ts** file. In the **constants.ts** file, we will add a **RTE_COMPONENT_CONFIGS**:

```
"RTE_COMPONENT_CONFIGS": {
  "ALL_COMPONENTS": ["tabs", "math", "collapsible", "skillreview", "image", "link", "video", "workedexamples"],
  "EXPLORATION_COMPONENTS": ["tabs", "math", "collapsible", "skillreview", "image", "link", "video"],
  "BLOG_COMPONENTS": ["image", "link", "video"]
},
```

This will help keep the all rte component display logic in one place and follow a uniform pattern everywhere.

We'll only be displaying the worked example component in the updated **Create New Study Guide Modal Explanation Content rte**, new **Study Guide Add Section Modal content rte**, [Create New Skill Modal explanation rte](#) and [Skill Concept Card Editor explanation rte](#).

In these files we are using the **schema based editor**. In the component files of the rtes, we will modify the **schema** to contain **ui_config:{rte_components: 'ALL_COMPONENTS'}**.

We will pass **{rte_components: 'EXPLORATION_COMPONENTS'}** for the exploration editor rtes and **{rte_components: 'BLOG_COMPONENTS'}** for the blog rtes.

This schema will be passed to the **schema-based-editor.component** which displays the editor based on the type. As the type will be html for all the above components, the **schema-based-html-editor** will be displayed. Make sure to update the [Schemas wiki page](#) after this is done.

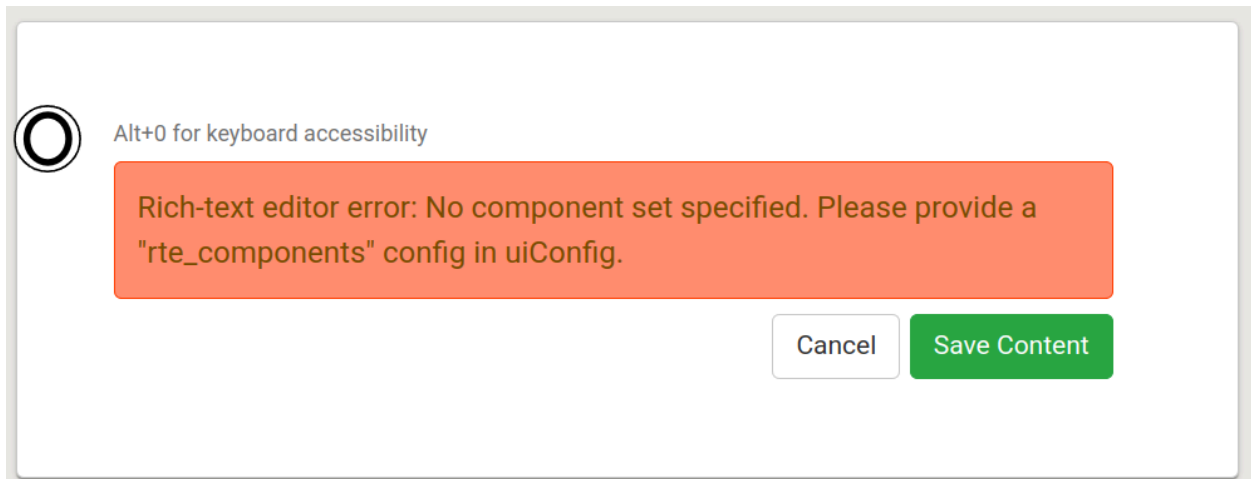
```
<schema-based-html-editor *ngIf="schema.type === 'html' && !schema.choices"
  [(ngModel)]="localValue"
  [disabled]="disabled"
  [labelForFocusTarget]="labelForFocusTarget"
  [uiConfig]="schema.ui_config"
  name="htmlEditor">
</schema-based-html-editor>
```

The field **uiConfig** of the **schema-based-html-editor** will be passed **{rte_components: 'ALL_COMPONENTS'}** and this will be passed to the **ck-editor-4-rte**. In [ck-editor-4-rte.component.ts](#) we will update the **UiConfig** to accept **rte_components**

```
interface UiConfig {
  (): UiConfig;
  rte_components: string;
  hide_complex_extensions: boolean;
  startup_focus_enabled?: boolean;
  language?: string;
  language_direction?: string;
}
```

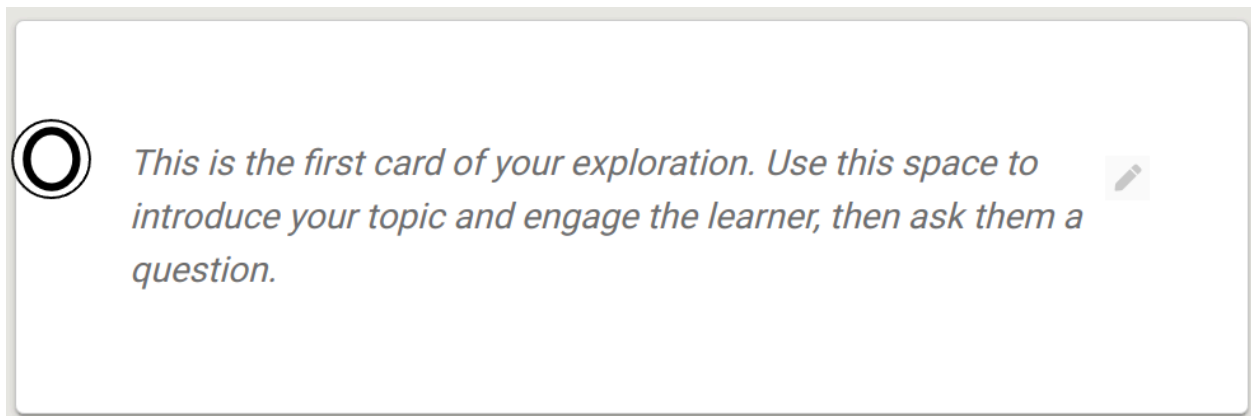
We will also update the **ngAfterViewInit** to call a method **getAllowedComponents** to display the **components based on the values passed by rte_components uiConfig**. If no value is passed, we will display a console error in addition to throwing an error.

Eg: If we do not pass an **rte_components** value in the exploration editor rte:

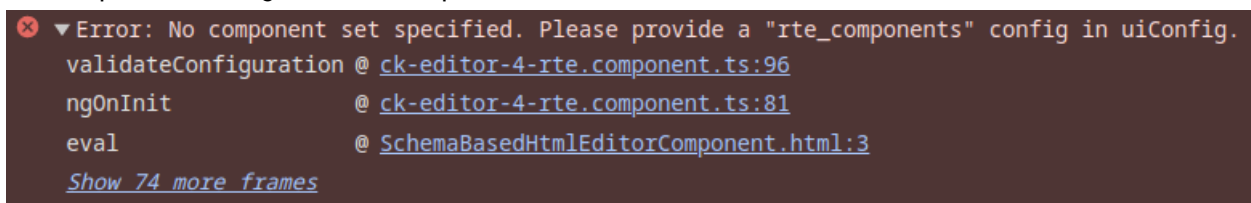


We cannot disable the Cancel/Save Content buttons as they are not managed by the ckeditor component. They are managed by the component using the ckeditor (exploration editor in this case). These buttons may or may not be present and can be different for different editors.

If Cancel/Save Content are clicked, we just go back to the preview card:



We don't see any rte components for the editor and see a warning message stating that the `rte_components` config has not been passed.



We also see the above console error.

After implementing these changes, we will be able to safely remove the [isInvalidForBlogPostEditorRTE](#) method in `ck-editor-4-rte.component.ts`.

In the editors that use the workedexample component, we will implement a variable **numberOfWorkedExamples** that will be incremented every time a workedexample is added and decremented when it is removed. If it becomes 2 we will not allow addition of any more worked examples. We will also implement character counting logic for the rte components (including worked examples) as they currently do not have any. (See [this section](#) for details on how this will be implemented.)

Link:

Customize This Component

The link URL. If no protocol is specified, HTTPS will be used.

The link text. If left blank, the link URL will be used.

The link text is too long. Please use at most 200 characters.

Cancel

Done

Collapsible:

Customize This Component

The heading for the collapsible block

Sample Header

The content of the collapsible block

B I [List Icon] [Link Icon] [Quote Icon] [Image Icon] [Table Icon]

[Text Icon] [Code Icon] [Media Icon] [More Icon]

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum]

The content length is too long. Please use at most 500 characters.

The content is too long. Please use at most 500 characters.

CancelDone

Tabs:

[illegible]

The worked example component will not be visible in unsupported rte. The user will not be able to copy or paste it there. When we will create the ck editor config for the different editors in [ck-editor-4-rte.component.ts](#), the worked example component will only be excluded from all editors except the Create New Study Guide Modal Explanation Content rte , new Study Guide Add Section Modal content rte, Create New Skill Modal explanation rte and Skill Concept Card Editor explanation rte. All the other editors will not have the ability to render the worked examples component.

Current behaviour when pasting unsupported rte components in the blog editor:

asdfasdfasdfasdf

We will fix this behaviour by adding paste validation logic to the [ck-editor-4-rte.component.ts](#) file and adding a div containing the error message and buttons based on the context in [ck-editor-4-rte.component.html](#).

If the user pastes only invalid content, we will show this:

BODY

A screenshot of a rich text editor interface. At the top is a toolbar with various icons for bold, italic, bulleted list, numbered list, link, unlink, and a 'Format' dropdown. Below the toolbar, a red error message box is displayed with the text: "The following component is not supported in this editor: skillreview. Please do not add it here." A "Dismiss" button is located at the bottom left of the message box. Below the message box is a large, empty text area for editing.

The user can then click on dismiss and continue using the rte.

If valid text/component is pasted with invalid content:

BODY

A screenshot of a rich text editor interface. The toolbar is identical to the previous screenshot. Below the toolbar, a red message box is displayed with the text: "The pasted content contains unsupported component: skillreview. Would you like to paste only the valid content?". There are two buttons: a green "Yes, paste valid content" button and a white "No, cancel" button. Below the message box is a large, empty text area for editing.

If user clicks yes:

BODY

A screenshot of a rich text editor interface. The toolbar is identical. Below the toolbar, the text "asdfasdfasdf" is pasted into the text area and is currently selected with a blue highlight.

The invalid component is stripped out and only the valid component/text is added.

If the user clicks no:

BODY

A screenshot of a rich text editor interface. The toolbar is identical. Below the toolbar, the text area is empty, indicating that the paste operation was cancelled.

The paste is cancelled.

Worked example will be added by clicking on the worked examples widget. On clicking it, an Add worked example modal will open.

Third-Party Libraries

No.	Third-party library name and version	Link to third-party library	Why it is needed	License (if third-party library)	[Android only] Min / target / max SDK version that the library supports
1	N/A				

“Service” Dependencies

No.	Dependency name	Why it is needed	What our plan is, if the dependency fails under us
1	N/A		

Key High-Level and Architectural Decisions

Decision 1: Selectively Displaying the Worked Examples RTE component.

We have considered the following alternatives:

1. Using the **ContextService**.
We would get the name of the current page from the service and display the Worked Example RTE component only for the skill and Study Guide editor pages opposite to what is currently being done for [not displaying certain components on the blog post editor page](#).
2. Using **uiConfig**
We will pass a string called **rte_type** in all the rtes using the **uiConfig** property. This will contain the type of components the rte has like 'ALL_COMPONENTS', 'BLOG_COMPONENTS', etc. The new **Create New Study Guide Modal Explanation Content rte**, new **Study Guide Add Section Modal rte**, **Create New Skill Modal rte** and **Skill Concept Card Editor rte** will pass 'ALL_COMPONENTS'.

Among these, we believe that Using **uiConfig** is the best approach, because:

- It provides more granular control as the specific editors for which the Worked Examples RTE component is to be used can just set the **rte_type** to 'ALL_COMPONENTS' while others can have their own custom rte component configuration.
- Minimal code changes will be required.
- The rte component display logic will become uniform over the entire codebase.

The above approaches are contrasted in detail in the following table:

	Alternative 1: Using ContextService	Alternative 2: Using uiConfig
--	-------------------------------------	-------------------------------

Control over which editors show the rte component	Less control as all editors on a page will show the component.	More control as only the specified components will show up in the rte.
Code Changes	Lots of changes to the context.service and component files of the pages will be required.	We will just need to update the constants to make any changes to what components to display and pass the correct enum to the uiConfig of the editors.
Uniformity	Custom implementation could be required for certain pages	Every editor on every page can use the same implementation to display different components

Risks and mitigations

Potential Risk	Mitigation
N/A	

Decision 2: Structure of the page_contents property of SubtopicPageModel

We have considered the following alternatives:

1. Add a sections property inside page_contents which will contain a list of headings and the contents.

```
page_contents: {
  sections: [
    {
      heading: SubtitledUnicode,
      content: SubtitledHtml
    },
    ...
  ]
}
```

2. Directly add a list of headings and contents to page_contents.

```
page_contents: [
  {
    heading: SubtitledUnicode,
    content: SubtitledHtml
  },
  ...
]
```

3. Changing the name of page_contents to sections and adding a list of headings and contents:

```
sections: [
  {
    heading: SubtitledUnicode,
    content: SubtitledHtml
  }
]
```

```

    },
    ...
  ]

```

Among these we believe Changing the name of page_contents to sections and adding a list of headings and contents is the best approach, because:

- It will make the code more readable and the structure of the sections easier to understand.
- In the future, if we decide to add properties unrelated to sections, we can create a separate property to store them.
- Allow for easy access to the sections property

The above approaches are contrasted in detail in the following table:

	Alternative 1: Add a sections property inside page_contents which will contain a list of headings and the contents	Alternative 2: Directly add a list of headings and contents to page_contents	Alternative 3: Changing the name of page_contents to sections and adding a list of headings and contents
Semantic Accuracy	Separates the concept of page_contents from sections which are a part of page_contents	Treats page_contents itself as a list of sections	Name directly reflects the actual content structure
Future Additions	It will allow easily adding other properties to page_contents alongside sections in the future	It is limiting as again the structure will have to be changed to accommodate the newer properties.	For future additions, new property will need to be created in the SubtopicPageModel as we will only be able to add section related data here
Access	Needs an extra step to access: page_contents.sections	Direct access: page_contents	Direct access: sections
Number of changes	Moderate	Lowest	Moderate

Risks and mitigations

Potential Risk	Mitigation
N/A	

Decision 3: Assigning Content IDs to the heading and content of sections

We will use a similar approach to **Explorations**. We will introduce a **next_content_id_index** property to the **StudyGuideModel** similar to the [next_content_id_index](#) of the **ExplorationModel**. We will use the

[ContentIdGenerator](#) from `translation_domain.py` to generate the `content_id` for the `heading` and `content` fields.

We will create the method `create_default_study_guide` to replace the `create_default_subtopic_page` method of `subtopic_page_domain.py` to create an instance of `ContentIdGenerator` called `content_id_generator` (we will not pass any parameters during the initialization so `next_content_id_index` will be 0) similar to the `create_default_exploration` method.

After this, `content_id_generator` will be passed to the `create_default_study_guide_section` method which will use it to generate a `content_id` for the `heading` and `content` fields. We will update the `ContentType` class to have a new content type called `SECTION = 'section'`. We will pass `translation_domain.ContentType.SECTION` as the first parameter and `'heading'/'content'` as the second parameter. It will increment the `next_content_id_index` to 1 after the heading id generation and to 2 after the content id generation.

We will return `content_id_generator.next_content_id_index` (which will be 2 after the increments) as the `next_content_id_index` as part of the newly created default Study Guide.

To add a new section, we will create a method called `add_section` (similar to `add_states`) in the `StudyGuide` class which will create an instance of `ContentIdGenerator`. This time we will pass `self.next_content_id_index` as a parameter during initialization so the generator gets initialized with the incremented value. We will pass this generator to the `create_default_section` method to create the section with the `content_ids` and finally update `self.next_content_id_index` to `content_id_generator.next_content_id_index`.

When a section is updated, no changes will be made to its or the other sections' `content_id`.

When a section is deleted, no changes will be made to the other sections' `content_id`.

If there is a section with heading `content_id` 0 and content `content_id` 1 and it is deleted and another section is added, the heading and content `content_ids` of the new section will be 2 and 3 respectively. If you go to the datastore and check there will be only 1 section in the sections list with heading and content `content_ids` as 2 and 3. This will also help in recognizing that a section has been deleted. For a Study Guide, a `content_id` once assigned will never be used again. `next_content_id_index` can ONLY be incremented.

Key Objectives for each Milestone

Milestone	Key Objective
Milestone 1	• Carry out the following migrations for the SubtopicPageModel: ◦ Convert the existing <code>page_contents</code> field into the new structure, a sections "repeated JsonProperty" field consisting of {heading: SubtitledUnicode, content: SubtitledHtml} dicts. ◦ Introduce a unique content ID for each translatable field, similar to explorations. • Update the study guide editor UI to accommodate the new structure, and the learner UI to match the Figma mocks • Deprecate the old <code>subtitled_html</code> field.
Milestone 2	• Verify that the existing <code>worked_examples</code> fields are empty in production. • Carry out a schema migration to safely deprecate the <code>worked_examples</code>

	field in the skill_contents part of the SkillModel. • Delete the worked_examples_count field from the SkillSummaryModel • Remove worked examples from the skill editor UI. • Implement a new 'Worked Example' RTE component that appears only in the skill explanation and study guide page RTEs. • Make sure to update the Schemas wiki page for the new schema for the rte components. • Validate that skill explanations cannot have more than 2 worked example components. • Ensure that the worked example rte component is translatable in the contributor dashboard. • Update the translation guide to include the worked example rte component. • (Carryover from M1) Move the written translations and voiceovers for study guide pages in EntityTranslationsModel and EntityVoiceoversModel, instead of within the SubtopicPage object (similar to explorations).
--	---

Implementation Approach

Subtopic Page Restructure:

[Web only] Storage Model Layer Changes

Create a new model called **StudyGuideModel**. It will have a **study_guide_id** similar to the **subtopic_page_id**, **topic_id**, **sections** similar to **page_contents**, **sections_schema_version** similar to **page_contents_schema_version**, **language_code**, **version** and **next_section_id_index** to store the id to be assigned to the next created section similar to the one used in explorations. The structure of the **sections** property:

```
"sections":[
  {
    "heading": SubtitledUnicode,
    "content": SubtitledHtml
  },
  {
    "heading": SubtitledUnicode,
    "content": SubtitledHtml
  },
  ....
]
```

[Web only] Storage Model Migrations

We will write a **PopulateStudyGuideSectionsJob** which will first get all the **SubtopicPageModels** in the datastore. For each SubtopicPage, **subtopic_page_id** will be copied to **study_guide_id**, **topic_id** will be copied as is. It will use the **subtopic_page_id** to get the **subtopic_title**. After this it will populate the **sections** field which will contain **heading** and **content** with their content_id as **section_heading_0** and

section_content_1 (this is the same as the **content_id** of a newly created study guide section using the subtopic editor) respectively. It will then extract the **html** of the **subtitled_html** field. The **unicode_str** of **heading** will be the **subtopic_title** and the **html** of **content** will be **html** of the old **subtitled_html** field. **sections_schema_version** will be set to 1, **language_code** will be copied as is, **next_content_id_index** will be set to 2 and **version** will be set to 1.

The current subtopic migration jobs will be maintained till the migration is complete. Once SubtopicPageModel is deprecated, subtopic migration jobs will be renamed to study guide migration jobs and the logic will be modified to work with the new structure having sections. Any future migrations will be done by creating a method `_convert_{studyguideproperty}_v1_dict_to_v2_dict` in the StudyGuide domain class and running the study guide migration jobs. (In the PR that makes this change, in the video proof, Mohit Ruwatia will show a trivial migration like changing the titles of all study guides to "Hello!".)

Domain Objects

We will create new domain object methods and services for the **StudyGuideModel** which will be similar to the **SubtopicPageModel** but will work with the new structure having sections. Both the models will be supported till the migration is complete. Any changes made to a **SuptopicPageModel** will be replicated for a **StudyGuideModel** object if it exists and on creation of a **SuptopicPageModel** object, a **StudyGuideModel** object will also be created. Once completed, the **SuptopicPageModel** and all its related domain object classes and services will be deprecated and only the **StudyGuideModel** will remain.

User Flows (Controllers and Services)

When the changes are saved on the topic editor page after a study guide is free from all warnings, the `/topic_editor_handler/data/<topic_id>` endpoint is called from the [updateTopic](#) method of the `editable-topic-backend-api.service.ts`.

The **topic_and_subtopic_page_change_dicts** will be renamed to **topic_and_study_guide_change_dicts**. It accepts `changeList` which is a list of **BackendChangeObjects**. We will modify it to use the old type **TopicSubtopicPagePropertyChange** as well as the newly created type for the **StudyGuide** called **TopicStudyGuidePagePropertyChange** in `change.model.ts` till the migration is complete. Then we will deprecate the **TopicSubtopicPagePropertyChange**.

(Till migration is going on:

```
TopicChange->TopicSubtopicPagePropertyChange|->TopicSubtopicPagePropertyChange  
|->TopicStudyGuidePagePropertyChange
```

Once migration is completed:

TopicChange->TopicSubtopicPagePropertyChange-|->TopicStudyGuidePagePropertyChange
 |->TopicSubtopicPagePropertyChange)

This will make sure that the same handler can be used for the **SubtopicPageModel** having the old structure and the **StudyGuideModel** having the new structure till the migration is complete.

This will call the `put` method of the `EditableTopicDataHandler` class of `topic_editor.py`

To populate the details on the study guide editor page, the `/subtopic_page_editor/data/<topic_id>/<subtopic_id>` endpoint is called from the `fetchSubtopicPageAsync` method which calls the `fetchSubtopicPage` method of

editable-topic-backend-api.service.ts. This calls the **get** method of the [EditableSubtopicPageDataHandler](#) class of **topic_editor.py**. This returns a [SubtopicPageBackendDict](#). Till the old ui exists, it will use this endpoint and methods. We will not use the values from the **StudyGuideModel** till all the data from the **SubtopicPageModel** has been migrated. We will create a new endpoint `/study_guide_editor_handler/data/<topic_id>/<study_guide_id>`, the method **fetchStudyGuideAsync** similar to [fetchSubtopicPageAsync](#) and **_fetchStudyGuide** similar to [_fetchSubtopicPage](#). **_fetchStudyGuide** will get the newly created **StudyGuideBackendDict** (having the same structure as [SubtopicPageBackendDict](#) except **page_contents** which will be replaced by **sections**). We will also create a **EditableStudyGuideDataHandler** (similar to [EditableSubtopicPageDataHandler](#)). The updated study guide editor page ui will use this. Once the updated ui has been created, we will deprecate the old endpoints and methods.

When the learner goes to view the study guide by going to the classroom, then clicking on the topic card and then clicking on the study guide card, a GET request is sent to `/subtopic_data_handler/<classroom_url_fragment>/<topic_url_fragment>/<subtopic_url_fragment>` from the [_fetchSubtopicData](#) method of [subtopic-viewer-backend-api.service.ts](#). This will call the [get](#) method of the [SubtopicPageDataHandler](#) of **subtopic_viewer.py**. It returns a response in the form of **ReadOnlySubtopicPageData** which will contain **pageContents** which will be of type **SubtopicPageContents**. We will keep the infrastructure like this and use it for the old ui. We will not use the values from the **StudyGuideModel** till all the data from the **SubtopicPageModel** has been migrated. We will create a new endpoint `/study_guide_data_handler/<classroom_url_fragment>/<topic_url_fragment>/<study_guide_url_fragment>`, **_fetchStudyGuideData** similar to [_fetchSubtopicData](#), [studyguide-viewer-backend-api.service.ts](#) similar to [subtopic-viewer-backend-api.service.ts](#), **StudyGuideDataHandler** similar to [SubtopicPageDataHandler](#), **study_guide_viewer.py** similar to **subtopic_viewer.py**. **StudyGuideDataHandler** will return a response in the form of **ReadOnlyStudyGuideData** which will contain **sections** which will be of type **StudyGuideSections**. The updated study guide viewer ui will use this. Once the updated ui has been created, we will deprecate the old endpoints and methods.

We will create a special method for android in the study guide domain file called **to_subtopic_page_dict_for_android** that will take a study guide domain object that will contain a list of sections, concatenate the headings and contents of each section into a single blob like the old subtopic page structure containing a single subtitled html field and return it with the other properties in the form of a dictionary. This is because the android side will still require a single field for the content of the study guide like it did earlier. This method will be used in [android.py](#) once the feature is in production. In **to_subtopic_page_dict_for_android** we will loop through all the sections of the study guide and add them to a list (We cannot directly convert the sections field to a string as it will be a list of dictionaries containing the 'heading' and 'content' keys). We will enclose all headings within a **<p> tag** and they will also have a ** tag** so that their content appears bold and the user experience is preserved. No processing will be needed for the content and they will be added to the list as is. The string will be like:

`'<p><strong>heading plaintext</strong></p>'` for headings

and

`'content html'` for content (the `<p>` tag will be present by default in the content html so we won't need to add it).

Then we will join the list with two newline characters into a single string as in the old structure, each heading, content and pair of heading and content were also separated by two newline characters as well .

```
concatenated_html = '\n\n'.join(concatenated_html_parts)
```

Finally the concatenated_html string will look like:

```
<p><strong>heading plaintext </strong></p>\n\ncontent
html\n\n<p><strong>heading plaintext</strong></p>\n\ncontent html...'
concatenated_html_parts will be the list containing all the sections' heading and content.
```

Finally, **to_subtopic_page_dict_for_android** will return this dictionary:

```
{
    'id': no changes to the domain object value,
    'topic_id': no changes to the domain object value,
    'page_contents': {
        'subtitled_html':
            {'content_id': 'content', 'html': concatenated_html}
    },
    'page_contents_schema_version': no changes to the domain object value,
    'language_code': no changes to the domain object value,
    'version': no changes to the domain object value
}
```

The old structure also contained recorded_voiceovers and written_translations as part of page_contents but we won't be including those as they were blank anyway and weren't used anywhere.

We will not pass the next_content_id_index value as it was not a part of the old structure of subtopic pages.

Note: We will append the output of the **to_subtopic_page_dict_for_android** method to the payload [here](#) and not call the to_dict method.

For android_test.py, we will add the tests **test_get_subtopic_returns_correct_json_for_multiple_sections** and **test_get_subtopic_returns_correct_json_for_single_section**.

When a topic containing a study guide will be published, a **GeneralTranslationOpportunityModel** will be created.

We will be using the general methods for opportunity creation for all models for the StudyGuideModel as well. These will be located in the [opportunity_services.py](#) file

- Whenever a topic is published, we will get all the study guides it has and create opportunities for them by calling the newly created **create opportunity method** in def **publish_topic** in **topic_services.py**.
- Whenever a topic is unpublished or deleted, we will get all the study guides it has and remove opportunities for them by calling the newly created **remove opportunity method** in def **unpublish_topic** in **topic_services.py**.
- We will also update the study guide opportunities every time a study guide or topic is updated.

When all translation opportunities for a particular language are to be fetched, the `/opportunitiessummaryhandler` is called from the [fetchTranslationOpportunitiesAsync](#) method of `contribution-opportunities-backend-api.service.ts`. This calls the `get` method of the class `ContributionOpportunitiesHandler` in `contributor_dashboard.py`. Here, `_get_translation_opportunity_dicts` is called which in turn calls `get_translation_opportunities` from `opportunity_services.py`. Currently, it only fetches exploration opportunities. We will add the logic to fetch subtopic page opportunities as well.

When translation opportunities for a particular entity are to be fetched, the `/gettranslatabletexthandler` is called from the [getTranslatableTextsAsync](#) method of `translate-text-backend-api.service.ts`. Currently the only parameters it accepts are the `exp_id` and `language_code`. We will modify it to accept a `subtopic_id` as well. This calls the `get` method of the class [TranslatableTextHandler](#) of `contributor_dashboard.py`. Currently it only fetches exploration models and gets the translatable text in them by calling the [get_translatable_text](#) method of `translation_services.py`. It currently only returns the exploration state names as keys with their content ids as values. We will modify it to support subtopic pages as well by returning the subtopic page ids as keys and their content ids as values in addition to the above.

[Web only] Web frontend changes

We will modify the **Create New Subtopic modal** to allow the addition of **1 Section** consisting of a **Heading and Content**. We will also add an **Add Section mat card** to the **subtopic editor page**. [Explained here](#).

We will also add character counting logic to the sections. We will set a limit of 600 characters as sections should be 100 words or less.

The average word is around 4.7 characters (excluding spaces) long for technical books, newspapers and journals according to these sites: [site 1](#), [site 2](#). As we are catering to learners and trying to communicate in simple language, I believe this should be slightly lower, that's why I came up with 4 characters (excluding spaces) and 6 characters (including spaces).

The logic will be similar to the **state content editor rte component** and [html](#) of the **Exploration Editor page**.

We will create a temporary form on the admin page to accept and reject study guide translations.

Documentation changes

Add tutorial on how to add sections to study guides in the wiki.

Update the wiki on how to translate sections using the form on the admin page.

Worked Examples RTE component.

[Web only] Storage Model Layer Changes

We will be deprecating the **worked_examples** sub property of the **skill_contents** property of the **SkillModel**.

We will also be deprecating the **worked_examples_count** property from the **SkillSummaryModel**.

[Web only] Storage Model Migrations

Check that all **SkillModels' skill_contents** have an empty **worked_examples** property.

If they are not empty, we will note down the contents of the worked examples, perform the migration and manually add them to the skill editor rte once the worked examples rte component is functional.

To migrate SkillModel, we will first update all the domain object and services to use the new structure without worked examples. Then we will create a new method **_convert_skill_contents_v4_dict_to_v5_dict**:

```
@classmethod
def _convert_skill_contents_v4_dict_to_v5_dict(
    cls, skill_contents_dict: SkillContentsDict
) -> SkillContentsDict:
    """Converts v4 skill contents to the v5 schema. The v5 schema
    removes worked examples completely from the skill contents.

    Args:
        skill_contents_dict: dict. The v4 skill_contents_dict.

    Returns:
        dict. The converted skill_contents_dict without worked_examples.
    """
    # Create a new dict without the worked_examples field
    converted_skill_contents_dict = {
        'explanation': skill_contents_dict['explanation'],
        'recorded_voiceovers': skill_contents_dict['recorded_voiceovers'],
        'written_translations': skill_contents_dict['written_translations']
    }

    return converted_skill_contents_dict
```

We will update the **CURRENT_SKILL_CONTENTS_SCHEMA_VERSION** property in [feconf.py](#) to 5. We will then run the **AuditSkillMigrationJob**. If it is successful, we will run the **MigrateSkillJob** to migrate the skills.

We can directly migrate the old **SkillSummaryModel** by removing all instances and uses of the **worked_examples_count** property from the codebase. This has been tested in [this doc](#).

Domain Objects

List of deprecations in **skill_domain.py**

- SKILL_CONTENTS_PROPERTY_WORKED_EXAMPLES
- UpdateSkillContentsPropertyWorkedExamplesCmd
- Class WorkedExampleDict
- Class WorkedExample

All the methods that use worked examples will be modified to work without them.

worked_examples will also be removed from **concept-card.model.ts** and **change.model.ts**.

worked_examples_count will be removed from **skill-summary.model.ts** and **augmented-skill-summary.model.ts**.

The **worked-example.model.ts** file will be deleted.

User Flows (Controllers and Services)

When a skill is updated, we send a PUT request to `/skill_editor_handler/data/<skill_id>` from the `updateSkillAsync` method of `skill-backend-api.service.ts` which calls the `put` method of the `EditableSkillDataHandler` class of `skill_editor.py`. The body of the request contains `change_dicts` which will no longer accept `worked_examples` as it will be modified in `change.model.ts`.

When a concept card is added to an rte (eg: Exploration editor rte) and viewed in a preview or player tab, a GET request is sent to `/concept_card_handler/<selected_skill_ids>` from the `fetchConceptCards` method of `concept-card-backend-api.service.ts`. This calls the `get` method of the `ConceptCardDataHandler` class of `concept_card_viewer.py` this returns an array with details of the selected concept card which will no longer contain `worked_examples` as they will be removed from `concept-card.model.ts`

[Web only] Web frontend changes

The worked example mat card will be removed from the skill editor page.

The Skill and Subtopic creation and editor rtes will include the new worked examples rte component as described in the [solution overview section](#).

Error warning box when rte_components is not passed:

We will add a new property called `configError` to `ck-editor-4-rte.component.ts` which will help to display an error box when the `rte_components` property is not passed.

We will add a method `validateConfiguration` to `ck-editor-4-rte.component.ts`:

```
1 private validateConfiguration(): void {
2   if (!this.uiConfig || !this.uiConfig.rte_components) {
3     this.configError = 'No component set specified. Please provide a "rte_components" config in uiConfig.';
4     console.error('Error: ' + this.configError);
5     return;
6   }
7
8   const rteComponents = this.uiConfig.rte_components;
9   const componentList = AppConstants.RTE_COMPONENT_CONFIGS[rteComponents];
10
11   if (!componentList) {
12     this.configError = `Component set "${rteComponents}" is not defined in AppConstants.RTE_COMPONENT_CONFIGS.`;
13     console.error('Error: ' + this.configError);
14     return;
15   }
16 }
```

We will call this method in the `ngOnInit` method to check validity when the component is initialized.

In `ck-editor-4-rte.component.html` we will add an `ngIf` to the main rte div that will make sure the rte only displays when there are no config errors and a div for the error which will only be displayed when there is a config error:

```
1 <div *ngIf="!configError">
2   <div class="d-flex flex-column-reverse">
3     <span class="oppia-cke-offline-warning" *ngIf="!connectedToInternet">* Tools with dark background can not be used when offline.</span>
4   </div>
5   <div contenteditable="true" class="oppia-rte-resizer oppia-rte e2e-test-rte" #oppiaRTE></div>
6 </div>
7
8 <div *ngIf="configError" class="oppia-rte-error-box">
9   RTE Editor Error: {{ configError }}
10 </div>
```

Character counting logic for the Collapsible, Image, Links, Tabs and Worked Examples rte components:
In [rte-helper-modal.component.ts](#):

We will create a new variable **CHARACTER_LIMITS** which will contain the allowed lengths for the various rte components. We will create a new method **getCharacterLimit** which will get the length according to the rte component. We will add another method **isContentLengthExceeded** which will perform the actual validation for the length. It will use the [computeHtmlLength](#) method of the **htmlLengthService** which has logic for taking into account the lengths of non text nodes. Formatting like bold, italics, bullet points etc won't affect the character count. This will be called in **onCustomizationArgsFormChange** to perform the validation anytime the values entered in the rte component change.

Documentation changes

Add tutorial on how to add worked examples to the skill and study guide editor pages in the wiki.

Add to [the translation guide](#) how to translate worked examples.

Metrics Plan

Reference: [PRD](#)

Event (see PRD)	Event parameters (see PRD)	Do we already record the event + parameters? - If so, please link to the corresponding code on GitHub. - If not, describe the changes needed to do so.
Learner selects the concept card option	Number of Learners that clicked on concept cards (L1)	We will create a new method in site-analytics.service.ts for this event called registerConceptCardSelectedEvent . This will be called any time a learner clicks on a concept card to open it.
Learner clicks 'see solution option'.	Number of learners that clicked on 'see solution option' (L2)	We will create a new method in site-analytics.service.ts for this event called registerConceptCardSolutionAccessedEvent . This will be called any time a learner clicks on the solution tab of the worked example of a concept card to open it.

Manually-collected metrics

Outcome	Metric	Steps for Measuring	What to report
Improved performance of the learner	The learner's number of mistakes and correct answers with and without the use of worked examples.	Facilitators will measure students' progress with and without the use of worked examples.	The learner's performance with and without the use of worked examples.

Implementation Plan

Milestone Table (include both PRs and other actions that need to be taken prior to launch)

Milestone 1

Key objective for this milestone: *Restructure study guides:*

- Carry out the following migrations for the SubtopicPageModel by creating a new StudyGuideModel:
 - Convert the existing page_contents field into the new structure, a sections "repeated JsonProperty" field consisting of {heading: SubtitledUnicode, content: SubtitledHtml} dicts.
 - Introduce a unique content ID for each translatable field, similar to explorations.
- Update the Study guide editor UI to accommodate the new structure, and the learner UI to match the Figma mocks
- Deprecate the old subtitled_html field.
- Verify that the existing worked_examples fields are empty in production.
- Carry out a schema migration to safely deprecate the worked_examples field in the skill_contents part of the SkillModel.
- Remove worked examples from the skill editor UI.

Milestone 1 table

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1	Rename all instances of Revision Card to Study Guide (#21725)	-	29 May	31 May
2	Add 'show_restructured_study_guides' feature flag for allowing the creators to access the updated study guide editor page and learners to access the updated study guide user interface (the actual content displayed by the study guides will be the same, just the user interface will be different).	-	31 May	31 May
3	Create the StudyGuideModel and create the new domain object classes for it • Rename the page_contents property to sections in the new model. • Create new domain object classes to use the sections property • Implement content_id assignment logic	-	3 June	6 June
4	Make changes to subtopic services and other files to use both the new and old structures. • If a SubtopicPageModel is created, a	3	7 June	9 June

	StudyGuideModel will also be created - Any changes made to a SubtopicPageModel will also be made to a StudyGuideModel Create a new study guide services file which will be used for the new ui and structure once the migration is complete and the new ui is created - Add methods for creation and updation of study guides and populating the values for the study guide viewer and editor pages.			
5	Create methods for the android controller that concatenate all the sections of a study guide into a single blob and return them	3, 4	8 June	9 June
–	Note: At this point, there will be two services files (subtopic and study guide services) and only one frontend (subtopic page). The old services file (subtopic services) will still be used for the current frontend. The new services file won't be used anywhere currently			
6	Write beam job to populate the StudyGuideModel	3, 4	9 June	16 June
–	Note: At this point, the data migration from the SubtopicPageModel to the StudyGuideModel is complete.			
7	(Needs feature flag) Make frontend changes related to the Study Guide Editor Page (along with acceptance tests) - Create the 'create new study guide' modal to accept heading and content - Implement the new section mat card - Add validation to check that a study guide always has at least 1 section - Implement character counting validation in the rte of the content field - Write acceptance tests for the curriculum admin	2, 3, 4, 6,	12 June	15 June
8	(Needs feature flag) Make frontend changes related to Study Guide Viewer Page (along with acceptance tests) - Update the existing study guide to reflect the new structure - Update the preview tab of the study guide editor page to show the same study guide that the learner sees - Write acceptance tests for the user	2, 3, 4, 6, 7	16 June	19 June
–	Note: At this point, there will be two frontends and services files in the codebase: Subtopic Page and Study Guides. Subtopic Page is the old frontend and will use the old services file while Study Guides is the new frontend and will use the new services file. Enabling the 'show_restructured_study_guides' feature flag will show the Study Guides frontend and disabling it will show the Subtopic Page frontend.			

9	Convert the subtopic page migration jobs file to study guide migration jobs. In the video proof, show a trivial migration like changing the titles of all study guides to “Hello!”	3, 4, 6, 7, 8	18 June	19 June
–	Note: At this point, there will still be two frontends. Use of the subtopic page model will be stopped at this point. Enabling the ‘show_restructured_study_guides’ feature flag will show the Study Guides frontend and disabling it will show the Subtopic Page frontend.			
10	Move the ‘show_restructured_study_guides’ feature flag to the test stage for testing.	2, 3, 4, 6, 7, 8, 9	20 June	20 June
–	<u>Demo to PMs/stakeholders:</u> • Creating a study guide with the new structure • Viewing a study guide with the new structure	2, 3, 4, 5, 6, 7, 8, 9, 10	20 June	
–	Note: At this point, the restructuring of the study guides is partially implemented (translations still left). User journeys of M1 can be tested by turning on the feature flag SHOW_RESTRUCTURED_STUDY_GUIDES.			
11	Add ‘enable_worked_examples_rte_component’ feature flag for allowing the creators to add worked examples to the study guide and skill editor rtes.	-	21 June	23 June
12	Write a mail to Sean to check there are no existing worked examples in the SkillModels in the datastore. If there are, he will make a note of them so that they can be added later and delete them manually.	-	23 June	24 June
13	Remove the Worked example mat card and its display logic from the concept cards.	-	25 June	26 June
14	Run migration jobs to remove the worked_examples field from Skill model	12, 13	27 June	4 July
15	Remove all use of worked_examples and worked_examples_count from the codebase • Remove all places where worked_examples is used in skill_domain.py, concept-card.model.ts and change.model.ts • Remove worked_examples_count from skill-summary.model.ts and augmented-skill-summary.model.ts • Delete worked-example.model.ts	14	29 June	4 July (after M1.15)
16	Make changes to the Skill model, Skill summary model and the affected domain object classes	14, 15	2 July	4 July

	- Remove worked_examples_count from the SkillSummaryModel by following this doc - Make changes to the backend skill services and other files to use the new structure			
-	Write a mail to Sean to confirm there are no existing worked examples in the SkillModels in the datastore (to make sure none have been added since the last time we checked)	-	4 June	4 June
-	Note: At this point, the backend restructuring of worked examples is complete.			
17	Bug fixes if required	2, 4, 5, 6, 7, 8, 9, 10	5 July	13 July
18	Move the 'show_restructured_study_guides' feature flag to the prod stage (after successful testing and approval from ProdOps Team) - Switch the android controller to use the new study guide method that converts the list of sections into blobs - Switch all domain layer calls to the study guides	2, 4, 5, 6, 7, 8, 9, 10, 17	13 July	14 July
-	Note: At this point, the new study guides should be used throughout the codebase. The infrastructure for the subtopic pages will still exist. We will remove it when we deprecate the feature flag.			
19	Break down the study guides on Oppia.org into sections and verify they are displaying correctly as currently they will have only one section with all the content: - Test this on the backup server first with a few study guides - Do this on prod after Sean grants the curriculum admin role	2, 4, 5, 6, 7, 8, 9, 10, 17	14 July	14 July
-	Note: At this point, the restructuring of the study guides is partially implemented (translations are remaining). User journeys of Milestone 1 can be tested by turning on the feature flag SHOW_RESTRUCTURED_STUDY_GUIDES. The restructured study guides should be accessible to all users and creators on the main Oppia site.			

Milestone 2

Key objective for this milestone: *Implement a new worked examples rte component:*

- Implement a new 'Worked Example' RTE component that appears only in the skill explanation and study guide page RTEs.
- Update the [Schemas wiki page](#) for the new schema for the rte components.

- Validate that skill explanations cannot have more than 2 worked example components.
- Ensure that the worked example rte component is translatable in the contributor dashboard.
- Update the translation guide to include the worked example rte component.
- Move the written translations and voiceovers for study guide pages in EntityTranslationsModel and EntityVoiceoversModel, instead of within the SubtopicPage object (similar to explorations).

No	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1	(Needs feature flag) Create the worked example rte component • Add the new property of rte_components to make component display logic uniform across all rtes • Implement logic for the new worked example rte component • Make sure the component previews and displays as expected • Make sure the component is editable	M1.11	19 July	22 July
2	Update the Schemas wiki page for the new schema for the rte components.	1	23 July	23 July
3	Enable translations for the worked example rte component • Enable copy tool usage for the worked example rte component • Make sure it is translatable in the contributor dashboard	1	25 July	27 July
4	Add a tutorial on how to translate worked example component in the translation guide	1, 3	28 July	28 July
5	Deprecate the 'show_restructured_study_guides' feature flag by removing all references to it (as the feature should be working as expected for 2 weeks by this point) • Change all file and component names with subtopic to study_guide. • Remove use of the subtopic page services file and deprecate it. • Deprecate the subtopic page model	M1.2, M1.3, M1.4, M1.5, M1.6, M1.7, M1.8, M1.9, M1.17, M1.18, M1.19,	30 July	31 July
–	Note: At this point, there will be only one frontend and services file on the codebase for study guides (the new one). The old ui, services, model, domain object and other files used by the old ui will be deprecated.			
6	Implement new rendering logic for rte components • Use the rte_components property for all rtes i.e. exploration editor, blog dashboard, etc.	-	2 August	3 August

	• Delete the custom blog dashboard rte component logic Note: There should be no user-visible changes introduced by this PR. This will be verified by checking the following: • The rtes work as expected for all components (make sure the rte_components config is passed everywhere) • The blog dashboard rte shows only the image, link and video component widgets like it did before (basically a user shouldn't be able to tell that this PR introduced any change to the component rendering logic of the rtes)			
7	(Needs feature flag) Add worked examples rte component to the skill and study guide editor pages rtes and write the acceptance tests • Only allow skill and study guide page editor rtes to use the worked examples component by using the rte_components property • Implement logic to only allow 2 worked examples in the selected rtes • Implement logic to not allow copy and pasting of the component in non supporting rtes • Ask Sean to re add any worked examples that existed before the changes to the SkillModel	M1.11, 1, 3	6 August	8 August
8	Move the 'enable_worked_examples_rte_component' feature flag to the test stage for testing.	M1.11, M1.13, M1.15, M1.16, M1.17, 1, 3, 6, 7	8 August	8 August
9	<u>Demo to PMs/stakeholders:</u> • Adding worked examples to skills and study guides • Viewing the worked examples • Translating the worked examples		8 August	
-	Note: At this point, the new worked examples rte component has been implemented. All the user journeys can be tested by turning on the feature flags SHOW_RESTRUCTURED_STUDY_GUIDES and ENABLE_WORKED_EXAMPLES_RTE_COMPONENT			
10	Add 'enable_study_guide_translations' feature flag for allowing study guide translation opportunities to be generated.	-	9 August	9 August

11	Use the GeneralTranslationOpportunityModel created by Hasitha for Study Guide Translation opportunities	M1.3, M1.4, M1.5, 10	11 August	13 August
12	Move the 'enable_worked_examples_rte_component' feature flag to the prod stage (after successful testing and approval from ProdOps Team)	M1.11, M1.13, M1.15, M1.16, M1.17, 1, 3, 6, 7, 8	14 August	14 August
13	(Needs feature flag) Create temporary form on the admin page to test study guide translations and voiceovers. Gate this form to never appear on prod (it can show up on test/backup servers).	M1.3, M1.4, M1.5, 10, 11	15 August	16 August
14	Modify TranslationOpportunityGenerationJob/ write new StudyGuideTranslationOpportunityGenerationJob along with the regenerate button on the admin page	M1.3, M1.4, M1.5, 10, 11, 13	18 August	25 August
15	Move the 'enable_study_guide_translations' feature flag to the test stage for testing.	M1.3, M1.4, M1.5, 10, 11, 13, 14	26 August	27 August
16	Deprecate the 'enable_worked_examples_rte_component' feature flag by removing all references to it (as the feature should be working as expected for 2 weeks by this point)	M1.11, M1.13, M1.15, M1.16, M1.17, 1, 3, 6, 7, 8, 12	28 August	29 August
17	Move the 'enable_study_guide_translations' feature flag to the prod stage (after successful testing and approval from ProdOps Team)	M1.3, M1.4, M1.5, 10, 11, 13, 14, 15	30 August	30 August
18	Bug fixes if required	M1.11, M1.13, M1.15, M1.16, M1.17, 1, 3, 6, 7, 8, 12	28 August	7 September
19	Deprecate the 'enable_study_guide_translations' feature flag by removing all references to it (as the feature should be working as expected for 2 weeks by this point)	M1.2, M1.4, M1.6, 8, 9, 10, 11, 15, 17	14 September	15 September

–	Note: At this point, the new worked examples rte component has been implemented. All the user journeys can be tested by turning on the feature flags SHOW_RESTRUCTURED_STUDY_GUIDES and ENABLE_WORKED_EXAMPLES_RTE_COMPONENT. The worked examples rte component should be accessible to all users and creators on the main Oppia site.
---	---

Feature Flags

#	Name of feature flag	What happens when the flag is turned on	Needs sync between Android / Web?	Key stakeholders
1.	SHOW_RESTRUCTURED_STUDY_GUIDES	Creators will be able to create study guides with sections and learners will be able to view them	Yes (to make sure that the new study guides display appropriately on the android app)	LaCE Team
2.	ENABLE_WORKED_EXAMPLES_RTE_COMPONENT	Creators will be able to add worked examples to the skills' review material section and the study guides' explanation	Yes (to make sure the worked examples rte component displays appropriately on the android app)	LaCE Team
3.	ENABLE_STUDY_GUIDE_TRANSLATIONS	Study guide translations will be accessible via a temporary form on the admin page	No	LaCE Team

Future Work

Note: This section is mainly for reference (since items listed here won't be part of the GSoC project). Proposals will primarily be evaluated based on the implementation plan above.

- Enable suggesting translations of study guide contents on the contributor dashboard
- Implementing the more detailed validation described in the [PRD](#) - for limiting the number of worked examples to 2 if there are no images (since such examples are harder for learners to parse), or limiting them to 3 if there are images